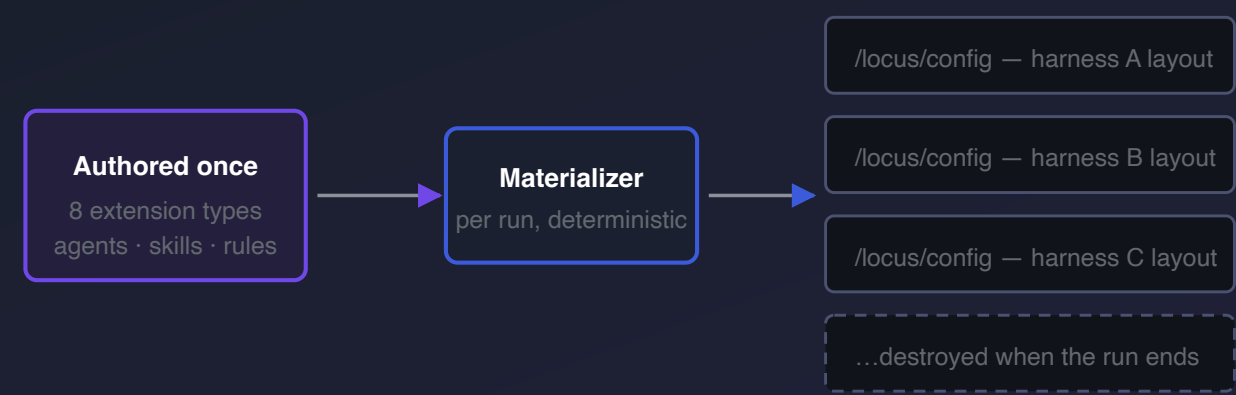


Locus

one surface, every harness

How the multi-agent, multi-harness IDE works inside and out — the runtime, the screens, the plugin contract, and the workflows that hold it together.



Contents

0 · Orientation

What Locus is, and is not
The three lifecycles
Vocabulary you need first

Part I · How it works internally

1.1 Process topology
1.2 Containers, images, networks
1.3 The git model
1.4 Credentials and egress
1.5 The one surface
1.6 Materialization
1.7 ACP and the event vocabulary
1.8 Sessions, runs, turns
1.9 The store and the fold
1.10 Memory
1.11 Token discipline

Part II · Using Locus

2.1 The shell and the address space
2.2 Setup
2.3 Plan
2.4 Manage
2.5 Interact
2.6 Review, Analytics, Memory
2.7 Dispatch and autorun
2.8 The inbox

Part III · Extending Locus

3.1 Extensions versus plugins
3.2 Writing an agent definition

3.3 The other seven extension types
3.4 The plugin contract
3.5 A harness plugin
3.6 A provider plugin
3.7 A CLI-tool plugin
3.8 A materializer
3.9 Trust and admission

Part IV · Workflows

4.1 The node vocabulary
4.2 Governance
4.3 Guardrails
4.4 The failure arbiter
4.5 Worked example

Part V · Operations

5.1 First-run setup
5.2 Backup and restore
5.3 Reconciliation and Stop all
5.4 Metrics that matter
5.5 Troubleshooting

Part VI · Current state

6.1 Milestones
6.2 Open decisions
6.3 Named risks

Appendices

A · Agent CLI reference
B · Glossary
C · Spec index

CHAPTER 0

Orientation

Locus is a desktop IDE whose primary surface is not an editor. It runs coding agents in disposable containers — one session per container — across many agent harnesses, and gives every one of them the same memory, the same mail, the same board, and the same tools.

What Locus is, and is not

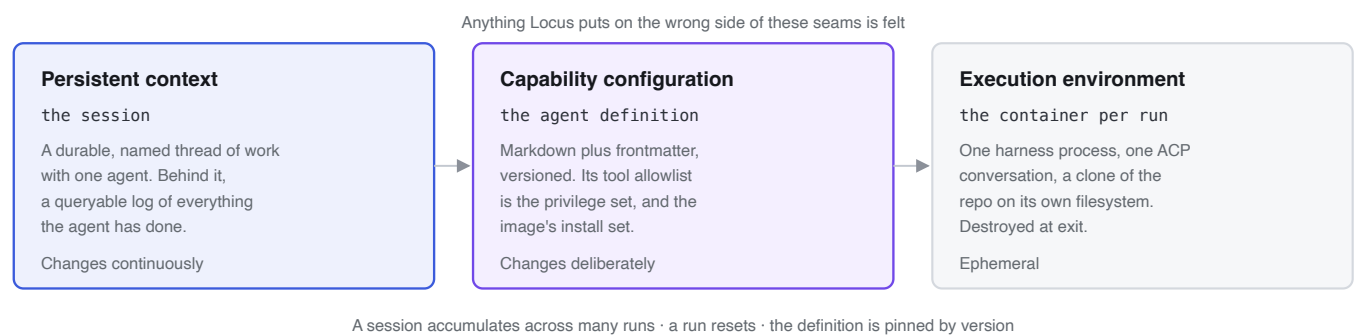
The useful map of the 2026 parallel-agent field has three tiers: **Tier 1** in-process subagents, **Tier 2** local orchestrators running agents in isolated workspaces on your machine, and **Tier 3** cloud async agents. Locus is **Tier 2, containing Tier 1, and deliberately not Tier 3.**

Nearly every Tier 2 tool solved *isolation*, because git worktrees made isolation easy. Almost none solved *coordination* or *shared learning*. Locus's bet is exactly there: shared memory and agent mail implemented once in Rust and available identically to every harness.

LOCUS IS	LOCUS IS NOT
A single-user local tool	A hosted platform or a team service
A control plane; the editor is a pane inside it	A VS Code fork or an extension host
One window, one selected project, cross-project views alongside	A window per project
Container-per-run, cloning from a bare local remote	Bind-mounted shared worktrees
A CLI surface for agents over a Unix socket	An MCP host. No MCP servers, ever
Postgres as the single store	A file tree of memory, mail, and wiki entries

The three lifecycles

Independent teams have converged on the same decomposition because the three parts have different lifecycles: context changes continuously, capabilities change deliberately, environments are ephemeral. Anthropic calls them Session, Harness, and Sandbox. Knowing which of the three you are touching tells you what a change costs.



The three lifecycles, and where Locus cuts them.

Vocabulary you need before anything else

TERM	MEANS
Project	Holds <i>one or more repos</i> and owns the board, wiki, and memory across all of them. Unrelated to GitHub Projects.
Harness	An agent CLI — the thing that talks to a model and runs a tool loop. Registered as a plugin. Pi is the only first-party one.
Session	A durable, named thread of work with one agent. Survives context resets.
Run	One container lifetime = one ACP conversation. A session has many runs.
Extension	One of eight authored content types (agents, skills, rules, ...) materialized into whatever a harness reads.
Plugin	An executable integration — CLI tool, harness, or provider — speaking JSON-RPC 2.0 over stdio.
Artifact	A structured deliverable an agent produces: a plan, a diff, a screenshot, a walkthrough. What you review instead of transcripts.
Workflow	A canvas graph that loops agents toward a goal, with governance and a mandatory <code>verify</code> .
Locator	<code>locus://<project>/...</code> — the one address space every navigation path resolves through.

THE THREE RULES CARRIED INTO EVERYTHING BELOW

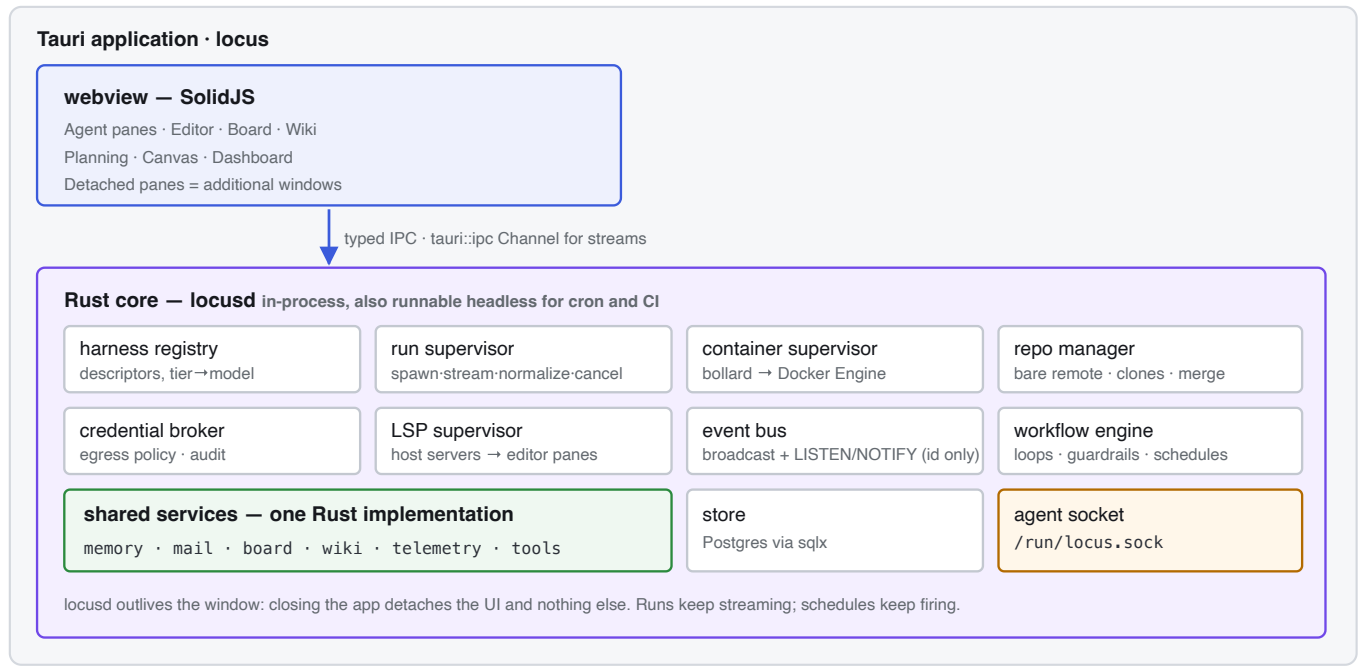
1. Nothing in `locus-core` names a harness — adding one is a trusted plugin manifest, never a `match` arm. 2. Every harness entry is complete; nothing is inherited, so a descriptor reads cold. 3. The database is derived where it can be — harness output, git, and the marketplace index are sources of truth Locus never writes.

PART I

How Locus works internally

1.1 Process topology

One Tauri application, one Rust core, one Postgres. The core — `locusd` — also runs headless, which is what makes a scheduled workflow that fires while no window is open a real thing rather than a promise.



Process topology. Every high-frequency path — session events, PTY bytes, diagnostics — is a `tauri::ipc::Channel`; only low-frequency notifications use the event system.

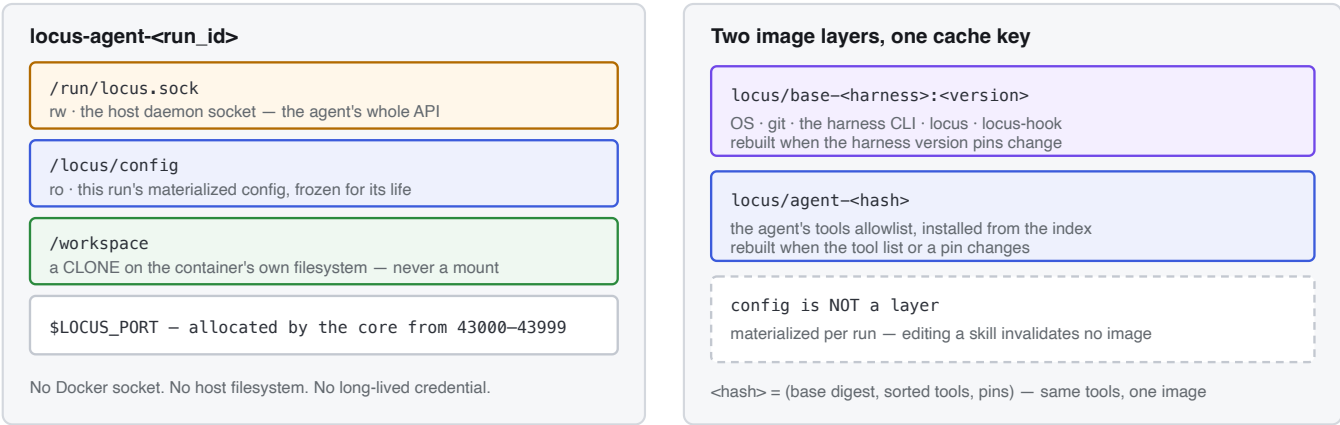
EVERY START RECONCILES

On boot the supervisor compares runs marked `running` against Docker. Container alive → re-attach its stream. Container gone → close the run as `aborted`, emit the event, file it in the inbox. Without this, a crash leaves rows claiming to be running forever.

1.2 Containers, images, networks

CONTAINER	LIFETIME	PURPOSE
<code>locus-postgres</code>	per machine	The store. <code>pgvector</code> + <code>tsvector</code> + window functions.
<code>locus-agent-<run_id></code>	per agent run	One harness process, one session. No TUI; ACP over stdio from the host.
<code>locus-svc-<project>-<name></code>	per project	Services the project declares — its own Postgres, Redis, the browser.

Network `locus-<project>` joins a project's agents and service containers. Agents reach each other and the project's services; they never reach another project. Only two paths are mounted into an agent container, and `/workspace` is not one of them.



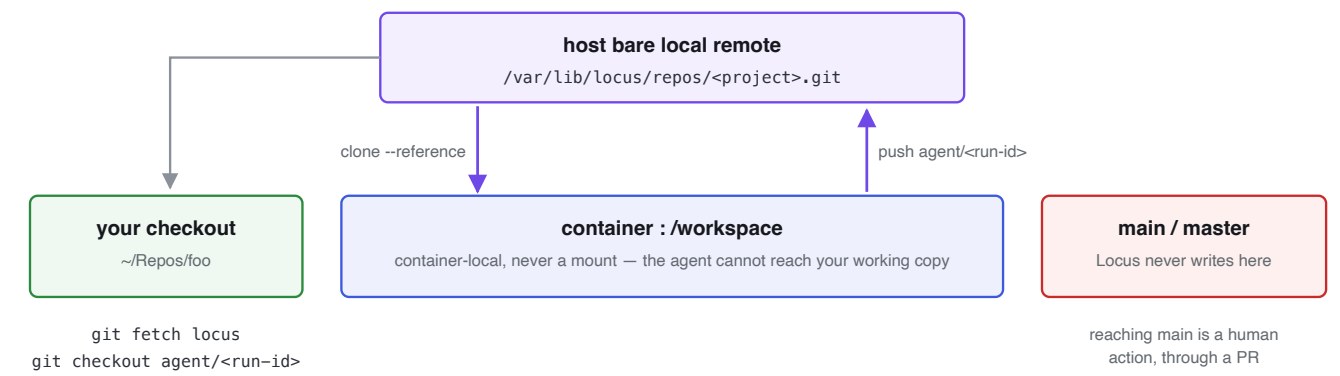
What reaches an agent container, and what a change costs. A cold build is minutes; a warm start is seconds.

TUI = FALSE IS REQUIRED, AND THE REGISTRY ENFORCES IT

A TUI harness multiplexes several sessions inside one process, which hides them from Locus — it could not display, count, or own them individually. A descriptor claiming `tui = true` is refused at registration. This preserves the 1:1 mapping between sessions and runs that everything downstream depends on.

1.3 The git model — a local remote, not shared worktrees

Every project has a **bare local remote** on the host. An agent container clones from it, works, commits, and pushes a branch back. You bring the work into your own repo with ordinary git.



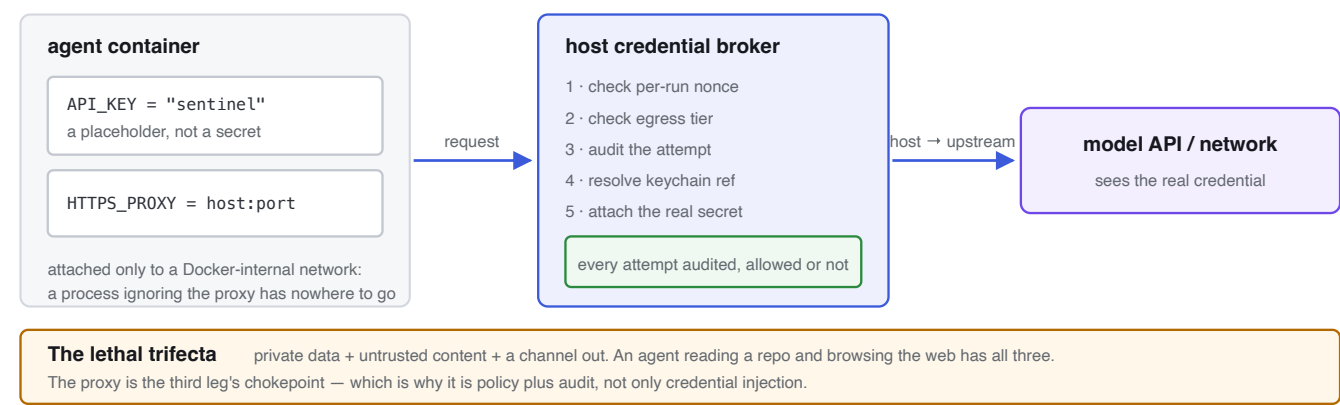
Isolation is real because the working copy was never mounted. A bind-mounted worktree can always be escaped by a path bug; a filesystem that does not exist in the container cannot.

Two honest costs. **Overlap surfaces at merge** — two agents can edit the same file and resolve it the way every team already does. **Clones cost disk and time** — mitigated by `--reference` against a shared object store, so N agents on one repo is not N copies of its history.

REPO MODE	WHERE THE CODE LIVES	YOU WORK THROUGH
Linked	<code>~/Repos/foo</code> stays yours; Locus syncs with it	your own checkout, and Locus
Managed	cloned from the forge, lives only inside Locus	Locus only

1.4 Credentials and egress

The requirement, stated before the mechanism: getting a harness authenticated inside a container must be *easy to use* and *secure*. The container must not hold a long-lived secret, and setting one up must not be a per-project chore. Spike 1 settled on a host proxy.



The container holds a sentinel; the OS keychain holds the secret; Postgres holds only the reference. That is the whole of the claim that Locus stores no credential.

THE DEFENSE LAYERS WORTH NAMING

LAYER	IN LOCUS
Privilege separation	The tool allowlist is the privilege set. A reviewer agent gets read-only tools and cannot write.
Sandboxing	Container per run, no host filesystem, no Docker socket, clone not mount.
Egress control	The proxy, with per-agent policy tiers.
Rate limiting	Cap tool calls per run, not only iterations and tokens — injection usually needs volume.
Canary tokens	A secret string in the materialized config that must never appear in output. If it does, the system prompt leaked.
Anomaly detection	Every run's tool sequence is already normalized in Postgres, so "unlike this agent's baseline" is a query.
Human in the loop	<code>Gate</code> nodes and <code>locus ask</code> , calibrated to what is <i>irreversible, touches production, or reaches credentials</i> .

STATED HONESTLY

Containers share the host kernel. This is weaker isolation than a microVM — Docker moved its own sandbox product to microVMs for exactly that reason. Containers are the pragmatic choice on Linux and macOS with no proprietary dependency, but the boundary is a kernel boundary, not a hypervisor one.

1.5 The one surface — eight extension types

An extension is authored **once**, in Locus. At run start the core reads the selected harness plugin's descriptor and writes that extension into the path and file format the harness expects, under `/locus/config`, for that run only.

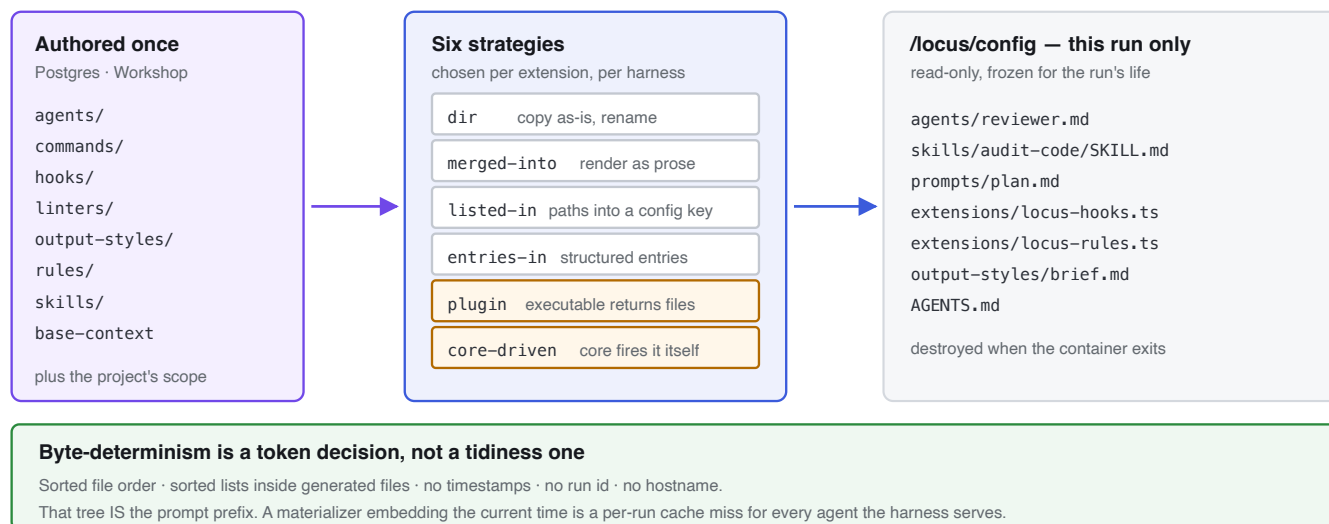
EXTENSION	IS	CONSUMED BY
<code>agents</code>	agent definitions — frontmatter over prose	the harness
<code>commands</code>	invocable prompt templates; worth writing only when they take arguments	the harness
<code>hooks</code>	shell scripts declaring their events; JSON in and out, exit 0 always	the harness
<code>linters</code>	a check plus the rule saying why, per directory	<code>locus lint</code> — not the harness
<code>output-styles</code>	how the agent writes; one is active per harness	the harness
<code>rules</code>	path-scoped instructions, loaded when a matching file is touched	the harness
<code>skills</code>	<code>SKILL.md</code> per directory, model-invocable	the harness
<code>base-context</code>	the always-loaded instructions every session starts with	the harness

Two of the eight do not behave like the rest, and every descriptor has to say so. **linters** are **tool-facing** — no harness reads them, so every harness supports them trivially and identically. **output-styles** is the **most divergent** — where a harness has no mechanism, the style is folded into `base-context` and the descriptor records the loss.

`base-context` is near-uniform in practice (`AGENTS.md` almost everywhere), which is why it also serves as the **fallback injection path** for harness plugins that cannot inject at session start.

1.6 Materialization — the code half of the contract

Declaring *where* an extension goes is not enough. A harness with no rules directory needs its rules turned into something it does read, and that is a transformation, not a path. Because Locus builds a whole config tree per run and destroys it after, materializers **generate whole files**: no markers, no merge, no prune, nothing to reconcile.



Materialization. The first four strategies are parameterized data and name no harness; only `plugin` requires an executable.

NAMING THE LOSS

A rule folded into always-on context is not the same thing as a rule the harness loads when a matching file is touched. An entry that hides the difference is how you end up believing in a capability you do not have — so a downgraded strategy carries `weaker_than_native`:

```
rules = { via = "merged-into", target = "context", strip_frontmatter = true,
  weaker_than_native = "always-on; path scoping is lost" }
```

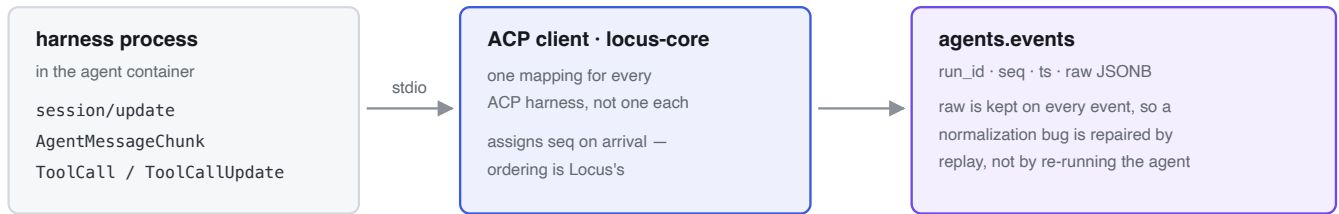
Native and downgraded counts shown anywhere in the UI are **computed from the registry**: registered harnesses × eight extensions is the denominator, and the numerator counts `weaker_than_native` keys. Registering a user harness moves the number without an edit.

1.7 ACP and the canonical event vocabulary

The **Agent Client Protocol** — JSON-RPC 2.0 over stdio, LSP-for-agents — is the *only* interface an agent session speaks to Locus over. One container hosts one ACP conversation, and the UI renders its normalized events. There is no terminal on the agent path; human terminals remain available for hand work only and are never agent sessions.

TWO CONSTRAINTS THAT SURVIVE THE CHOICE

`session/new` accepts only `cwd` and `mcpServers`, so a client cannot inject a system prompt. Prompt assembly belongs entirely to the harness; base-context, skills, rules, and tools reach the agent through the materialized config tree instead. And `mcpServers` is passed empty — always.



The closed twelve-verb vocabulary — every source normalizes to exactly this set

session_start user assistant thinking tool_call tool_result tool_error
permission_request subagent_start subagent_stop aborted session_end

A missing verb is recorded as missing, never synthesized. Token usage is an attribute — usage{input,output,cache_read,cache_write} — never a verb.

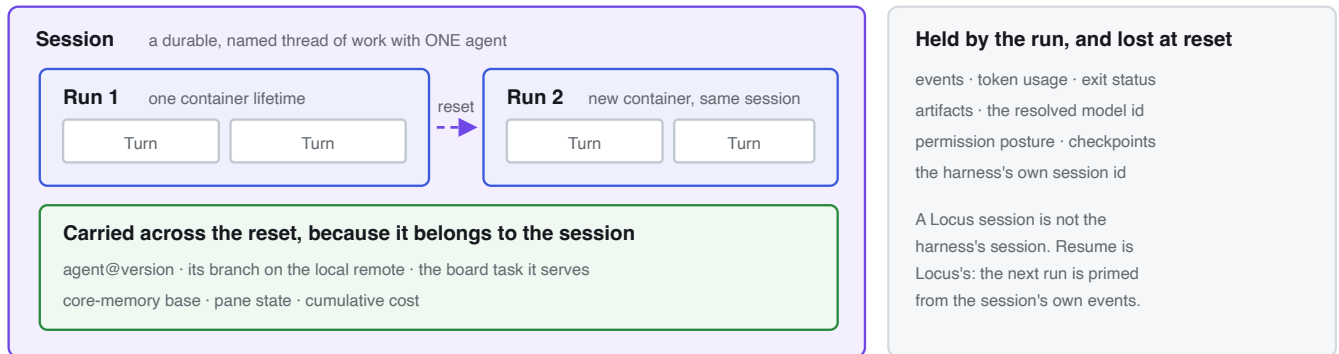
The event vocabulary is how Locus is tested: "run this, assert these events appeared," identical across the first-party Pi harness and any trusted user harness plugin, with no test-only instrumentation.

PERMISSION_REQUEST IS POSTURE-AWARE

In the default **bypass** posture every harness gate is switched off at launch, so a `permission_request` is a *misconfiguration alarm*. In a dispatch-selected **gated** posture the same event is a visible human-action request that blocks until resolved. The run's stored posture, not the verb alone, determines its meaning.

1.8 Sessions, runs, turns

"Session" is overloaded — ACP has sessions, every harness has sessions, the UI has panes. Locus needs one definition, and the split exists for exactly one reason: **the session is what survives the reset**.



The Ralph loop the field converged on — pick, act, validate, commit, RESET — needs something that persists across resets and something that does not.

The run is the thing that resets. The session is the thing that accumulates.

Session, run, turn. A workflow iteration ends a run and starts a new one in the same session.

HANDOFFS — A PAYLOAD, NOT A TRANSCRIPT

Guardrails kill and reassign after three stuck iterations, and a session belongs to exactly one agent. Work therefore changes hands regularly, and without a payload the successor inherits a branch and a task and rediscovers everything else. `locus handoff <agent> --why ...` ends the current session and opens a new one on the same task and branch, carrying:

FIELD	HOLDS
<code>goal</code>	what this work is for, restated
<code>done[]</code>	what is finished, each with the evidence

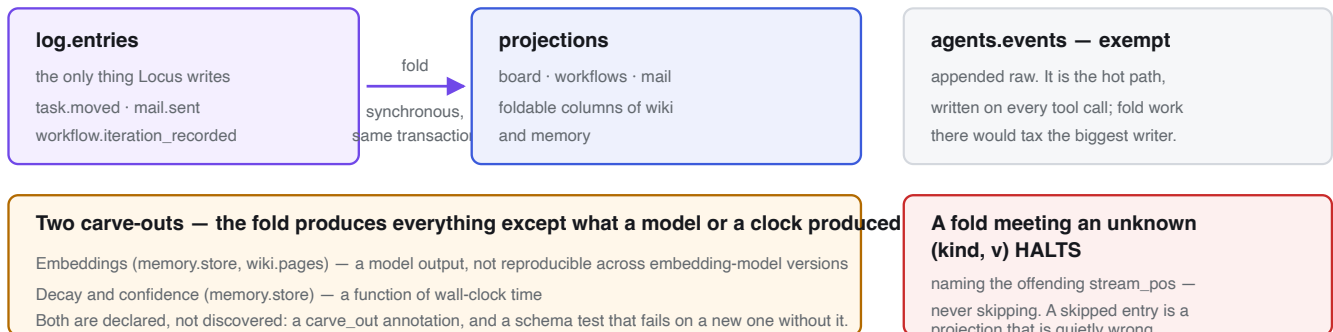
FIELD	HOLDS
<code>remaining[]</code>	what is not, in the order it should be taken
<code>attempted[]</code>	what was tried and did not work — the expensive half, and the reason the mechanism pays for itself
<code>decisions[]</code>	choices already made, so they are not re-litigated
<code>open[]</code>	questions the successor inherits

A handoff is *not* mail and *not* `locus agent invoke`. Mail is a message between agents that both keep working; invoke is a nested run that returns to its caller. A handoff transfers ownership and does not come back.

1.9 The store, and the fold

Postgres is the single source of truth, in nine schemas. Harness output, git, and the marketplace index are sources of truth *outside* Locus and are never written by it. Board, wiki, memory, and mail are the exception — they exist only in Postgres, which is why backup is a requirement rather than an afterthought.

SCHEMA	HOLDS
<code>core</code>	projects, repos (multi-repo per project), local remotes, settings
<code>agents</code>	versioned <code>agent_defs</code> , sessions, runs, parent/child run edges, normalized events, artifacts and their comment threads
<code>board</code>	tasks (fixed columns, blocked as a status), dependency edges, transitions, assignments, task↔run links, evidence, linked forge issues
<code>wiki</code>	typed pages, revisions, links, contradictions, ingest log, embeddings
<code>memory</code>	core (bounded, materialized per run) and store (facts, scope, provenance, embeddings, confidence, decay)
<code>workflows</code>	versioned <code>workflow_defs</code> (<code>graph</code> + <code>spec</code> JSONB), schedules, executions, iterations, guardrail trips, verify results
<code>mail</code>	threads, messages, delivery state
<code>market</code>	manifests, installs, per-image tool sets
<code>log</code>	<code>entries</code> — Locus's own domain event log; the only table any of the above is written <i>from</i>



locus rebuild regenerates every projection. It does not touch the carve-outs, because they were never derived from the log.

Two logs, one ordering: agents.events keeps its closed 12-verb enum; log.entries carries an open, per-kind versioned payload. They share one stream_pos per project.

Event sourcing with two named exemptions. The fold is synchronous in the append's own transaction, so a projection is never stale — available only because the core is the sole writer.

1.10 Memory

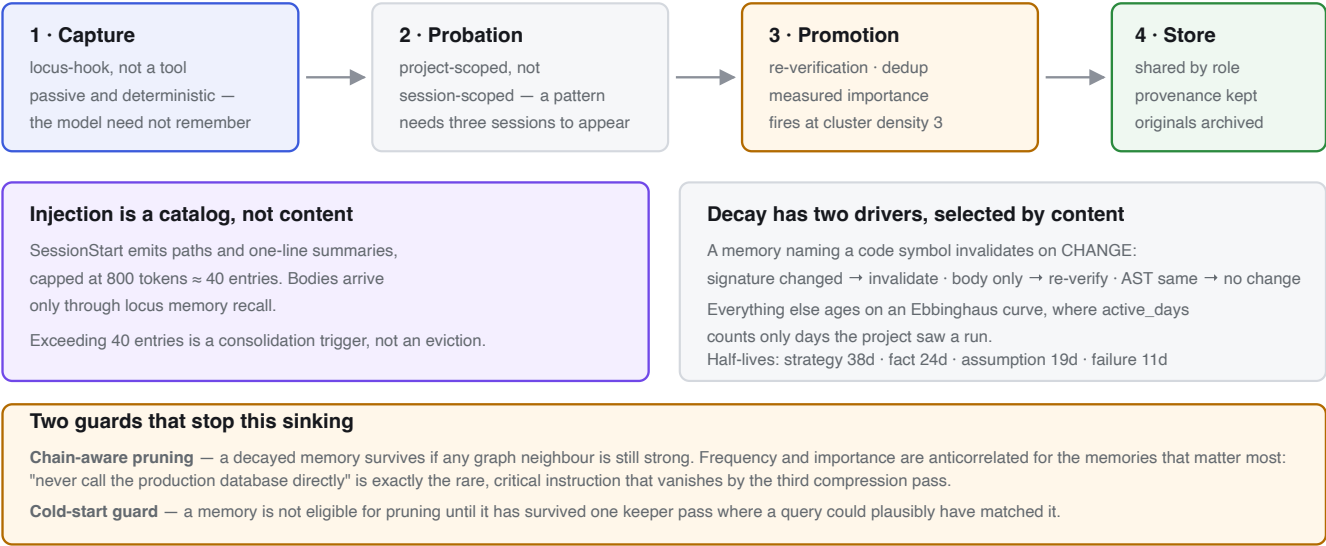
Four layers, split by **lifetime**. Locus owns three; the one it does not own is the one every memory product tries to build twice.

LAYER	LIVES IN	DIES WHEN	OWNER	SHARED
Working	the context window	the run ends	sub-harness	no
Short-term	probation buffer	promoted, or aged out	Locus	no
Long-term	Postgres	decays below threshold	Locus	yes, by role

LAYER	LIVES IN	DIES WHEN	OWNER	SHARED
Written	git — artifacts	never	the repo	everyone

THERE IS NO SHARED SHORT-TERM

Two agents sharing scratch state is a documented failure mode: two writers sharing one home compound each other's entries into state neither of them authored. Sharing begins at long-term, *after* consolidation, so the promotion boundary and the trust boundary are the same line.



The memory pipeline. The keeper is an ordinary agent definition running at `high` tier on genuine project idle; the primary agent has no memory-edit tools at all.

RECALL DEPTH REVERSES BY TASK CLASS

This is the counter-intuitive result the design is built around. On factual QA, accuracy rises monotonically with *k*. On agentic tasks the sign flips — success falls from 32.1% at *k*=1 to ~25% at *k*=5, and flat retrievers drop *below* the 22.4% no-memory baseline. Over-retrieved agents do not simply fail; they wander.

TASK CLASS	SUBSTRATE	K	GRAPH EXPANSION
Code generation / edit	flat, similarity only	1	off
Planning / strategy	distilled procedural cues	1	off
Research / review	structural, hierarchical	high	on

`task_class: code | plan | research` sits on the agent definition, defaulting to `code` ; a workflow's `Agent` node overrides it per role. Turn-level body injection is `off` for `code` and `plan` — a missed injection costs one tool call, while a wrong one measured below the no-memory baseline.

MEMORY IS NOT THE WIKI

The wiki is curated prose a human reads and reviews. Memory is what an agent recalls: scoped facts with provenance, embeddings, and decay. They share `pgvector` and nothing else. Two more kinds of knowledge are deliberately *not* memory: code structure (AST-derived, queried live via `codanna`) and the project constitution (small, always loaded, and human-written — LLM-written context files measure ~3% worse at 20%+ more cost).

1.11 Token discipline

If token usage explains most of the variance in how a run goes, tokens are not an operational concern that gets attention when a bill arrives — they are a design constraint on every surface that puts bytes in front of a model. Four levers, in the order they pay.

1 · Prefix stability — cache rate is the number

Cached reads bill at a fraction of fresh input, so a long session with a stable prefix is cheaper than a short one that busts the cache every turn.

- materialization is deterministic · the config tree is frozen for the run
- the memory catalog is a snapshot · mutable content goes LAST

Below ~80% on a long session means something in front is moving.

2 · Prevention at the tool boundary

The cheapest token is the one that never enters context. A PreToolUse hook in the base image rewrites verbose commands and compacts their output before the result is appended — 60–90% reported savings on ordinary dev operations, needing no cooperation from the agent.

Rule it enforces: a tool high on bytes but low on calls is returning too much per call.

3 · Diagnosis, as a query

Rank tools by how much result payload they put into history — halving the biggest contributor beats eliminating three small ones. Every tool_result is already a normalized row, so that ranking is a GROUP BY per agent, per project, per harness.

It is what makes "this agent is expensive" actionable rather than merely true.

4 · Summaries with handles, never bodies

memory → 800-token catalog → locus memory recall
tool docs → one line, ~15 tokens → locus tools docs <name>
big tool results → summary + id → locus artifact get
artifacts → OCR text or a frame → locus artifact get

Nothing is pre-loaded that can be fetched — which also dissolves over-retrieval.

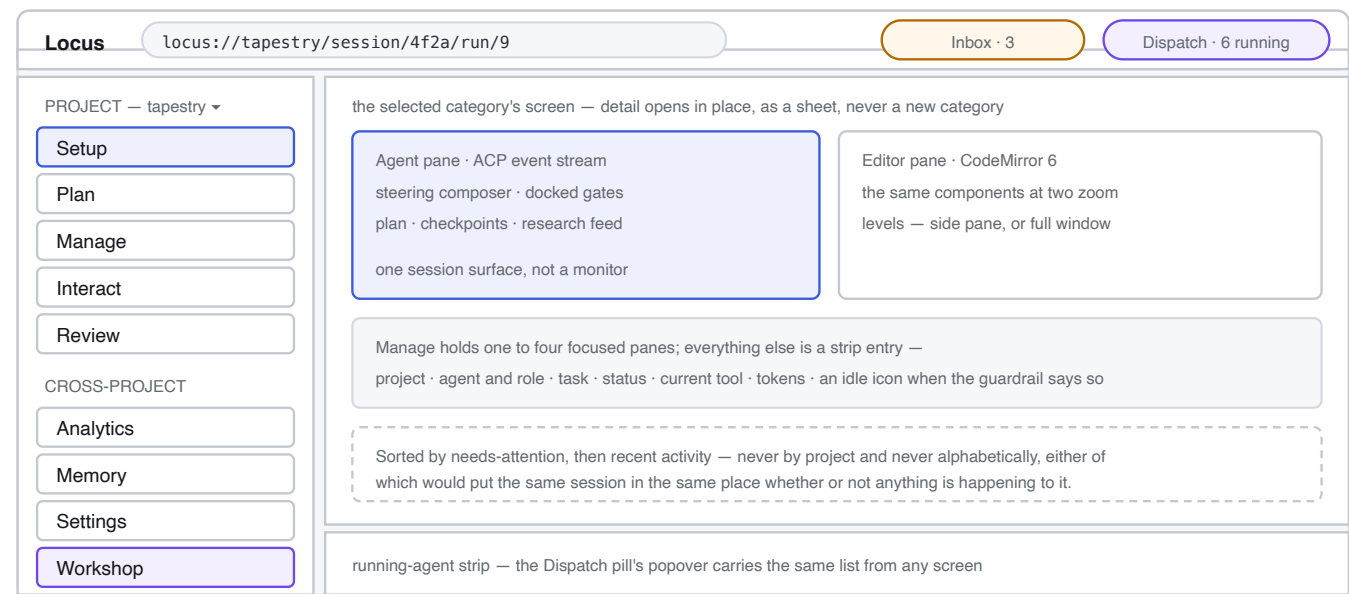
One asymmetry aims the effort: **output tokens cost roughly five times input**. That is what the `output-styles` extension is actually for.

PART II

Using Locus

2.1 The shell and the address space

One window, one selected project, and a group of views that are explicitly not scoped to it. A window per project would rebuild the thing the design exists to remove: one inbox per window, one place per window to notice an agent has been idle for an hour.



The shell. Inbox and Dispatch are pills rather than rail entries because "does anything need me?" and "is anything happening?" are questions you ask from every screen.

NINE CATEGORIES, IN TWO GROUPS

CATEGORY	HOLDS	THE QUESTION IT ANSWERS
Setup PROJECT	harnesses, repos, extensions, CLI tools, base context; what this project has kept; its numbers	how is this project configured, and what does it know
Plan PROJECT	the planning module — interviewer, researcher, auditor; the spec and the recommendation	what should we build, and is it understood
Manage PROJECT	the board and its sessions, as kanban, list, dependency graph, or timeline	what is assigned, and what is running
Interact PROJECT	sessions that are yours alone — a container, a branch, and an agent you talk to directly	the hands-on work, mine
Review PROJECT	tests, linters, LSP diagnostics, and agent reviews for this project	what is wrong right now
Analytics	spend, tokens, cache, run times, task outcomes, memory and extension usage	what happened, and was it any good
Memory	short-term, long-term, artifacts, the wiki, and mail	what do we already know, and what disagrees
Settings	guardrails and everything else that is per install	what are the defaults

CATEGORY	HOLDS	THE QUESTION IT ANSWERS
Workshop	Plugins: CLI Tool, Harness, Provider. Extensions: the eight types, agent definitions, Workflows	the tools themselves

MANAGE OPERATES; WORKSHOP AUTHORS

An agent you are assigning work to and an agent definition you are editing are different activities on the same object — one is a thing doing work, the other is a document under version control. The same holds for workflows: the canvas is Workshop, a running execution is Manage. That is why agents and workflows appear twice in the rail and it is not a duplication.

ONE ADDRESS SPACE, SO THERE IS ONE RESOLVER

```
locus://<project>/session/<id>[/run/<id>]
locus://<project>/task/<id>           artifact/<id>           page/<slug>
locus://<project>/workflow/<id>[/execution/<id>]           agent/<name>@<version>
```

The command palette, global search, inbox items, board-card links, artifact comments, notification deep links, and a detached window's identity are otherwise seven navigation paths that drift apart. Against one locator they are one resolver with seven callers. `Cmd-K` resolves a locator; `Cmd-P` searches for one; back/forward per window is a stack of them. `resolve(locate(v,p)) == (v,p)` is asserted, not assumed.

2.2 Setup — configuring a project

Three tabs: **Settings**, **Persistence**, **Analytics**.

SETTINGS — FIVE PANELS

PANEL	WHAT IT CONTROLS, AND THE RULE THAT BINDS IT
Harnesses	An <i>ordered</i> allow-list, not a set: enabled harnesses are offered to the router in the order listed, and anything the router does not claim runs on the agent default. A harness with no working adapter is listed but not selectable . Exactly one row carries the agent default, and it must itself be allow-listed. The picker draws only from harness plugins configured in Workshop → Plugins.
Repos	A repo belongs to exactly one project. Reassigning re-tags new activity; the old project tag stays on the historical record, so history never silently changes project.
Extensions	Seven toggle groups pulled from Workshop's defaults, each with a tri-state master and per-item toggles. Switching one off removes it from the <i>next</i> materialized tree — it does not delete the authored definition. The scope contract is subtraction-only.
CLI tools	Catalog search on the left, "in this project" on the right. Adding or removing rebuilds the project's image once , not per run. A tool an agent cannot find is the second most common reason a run stalls.
Base context	Exactly one <code>base.md</code> , and there is no second. A token-budget meter, version, edit time, run count, History and Save. Over budget reads as "something belongs in a skill or a rule instead" — a design signal, not an error.

PERSISTENCE — EVERYTHING THE PROJECT HAS KEPT

GROUP	SECTIONS	DELETE OFFERED
Memory	Short-Term · Long-Term · Artifacts	Long-Term and Artifacts only

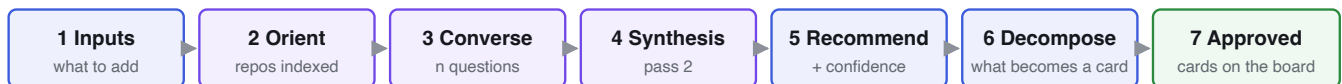
GROUP	SECTIONS	DELETE OFFERED
Specs & Tasks	plan bodies with nested task lists that navigate to the board	none
Research	source and synthesis items	none

Short-Term carries no delete because it is already governed by its own decay; Specs and Research carry none because they are the written record git owns. Long-Term and Artifacts are the two places where a user action, not decay or git, is the only way a record goes away.

2.3 Plan — a guided conversation that produces a reviewable spec

Three agents, not one. The split is structural: measured on ProgramBench, single-pass agents terminate early by their own choice at a median of 22–177 turns, because exploration and implementation compete for the same attention and implementation wins. Splitting them lifted test pass rates 6.9–21.3% relative.

AGENT	TASK_CLASS	JOB
Interviewer	plan	Drives the questions, holds the state, writes the artifacts
Researcher	research	Facts, prior art, feasibility. Never asked for intent
Auditor	plan	Fresh context, adversarial. Grades the spec it did not write



Stage 3 · the question loop

ask you — intent, priorities, trade-offs
 dispatch researcher — fact, feasibility, prior art
 scope decision → you, inline, not a separate gate
 drop — answered, or irrelevant to the goal

Stage 4 · synthesis is two passes, and the second SUBTRACTS

pass 1 completeness — types, state transitions, edge cases, trust boundaries, error conditions
 pass 2 reduction — cut what is unsupported, rewrite what is ambiguous. Downstream agents treat an unsupported requirement as mandatory.

The properties that make this survivable

- **The goal is an input, not an output.** Topics are ranked by how much they bear on it, and the interview ends when nothing unresolved still does.
- **Requirement ids are stable (R-05).** A card already on the board keeps pointing at the requirement it came from, even after a rewrite.
- **Scope changes are approvals, never automatic** — both directions. A silent narrowing ships something missing and leaves no trace of which step removed it.
- **Effort scales by a ratchet** — starts minimal, escalates on evidence, never de-escalates. A one-line fix stays three questions.

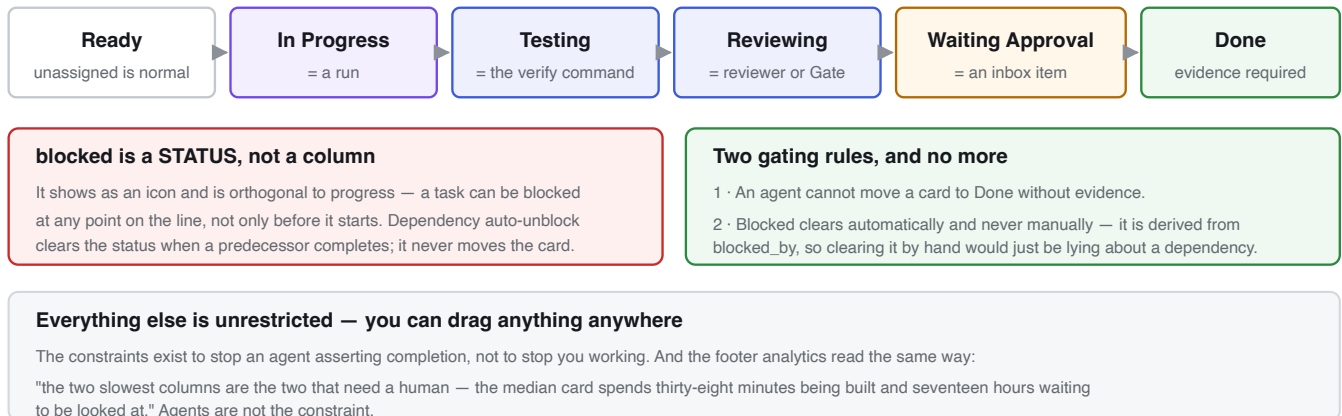
The seven stages. Audit is not one of them: the auditor grades continuously and its findings arrive as `open[]` attached to the clause that has the gap.

RE-PLANNING KEEPS SPEC AND TASKS IN SYNC

TASK STATE WHEN ITS REQUIREMENT CHANGES	RULE
Not started	rewrite in place
In progress	flag it and notify the session — never silently mutate a task an agent is working from
Done	never touch. Emit a <i>new</i> task for the delta, linked to the original
Requirement deleted	close as <code>superseded</code> , never delete — the trace has to survive

2.4 Manage — the board and its sessions

Fixed columns across every project, deliberately not configurable, and only the gating something else already depends on.



The board. Four of the six columns map onto machinery already in the design, which is why the column set is fixed rather than configurable.

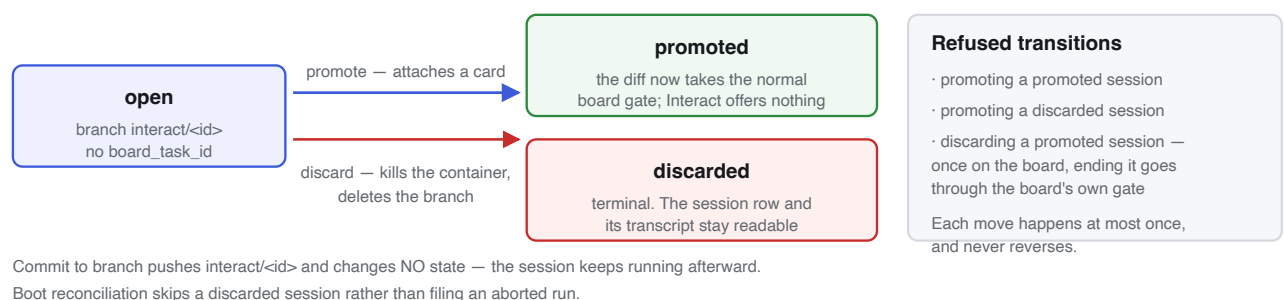
Manage offers four views over one toolbar. **Kanban** carries card decoration sourced from the task fold and its evidence — a live pulse with the running token count, a blocked marker naming the gate, a `stuck 3/3` ring from the guardrail, the verify command on Testing cards, the gate name on Reviewing cards, and the evidence summary on Done. **List** pairs active and inactive sessions with a live detail pane, including the stuck-guardrail banner and its **Hand off** control. **Graph** draws the dependency DAG, where every edge is one `board.task_dependencies` row and there is no edge with no row. **Timeline** is the same tasks against wall-clock.

EVIDENCE PROVES THE REQUIREMENT WAS MET, NOT THAT THE FEATURE IS RIGHT

A deployment report describes a payments backend that passed CI in two cycles against an adversarially-written suite and still shipped two business-logic failures, because neither behavior was in the contract. No amount of verification reaches outside its requirement. This is why the planning module's *elicitation*, not its audit, is where the quality actually comes from.

2.5 Interact — sessions the board never sees

A container, a branch, and an agent you talk to directly. No card, no plan, no gate — the escape hatch every plan-driven system needs, and honest about its cost: nothing there is reviewed, so nothing reaches `main` from it until you promote it or land the branch yourself.



The Interact session state machine. A session with no board task is a first-class named state, not a null `board_task_id` the rest of the system has to interpret.

2.6 Review, Analytics, and Memory

Review is now; Analytics is after. Review says what is broken in *this* project — a failing test, a linter violation, an LSP diagnostic, a reviewer's finding. It is a scheduled or manual aggregated pass, never a stream: sending a finding to the inbox tracks it as a to-do and leaves it listed where it was found. Keeping the two apart stops Review growing into a query tool and stops Analytics growing a live view nobody watches.

Analytics ignores the project filter entirely, because cross-project questions are the useful ones and a spend chart that silently showed one project would be worse than no chart.

Memory is where the four knowledge homes become inspectable:

SCREEN	SHOWS
Short-term	The context window as an anatomy, not a cache: "nothing here is stored — it is rebuilt from scratch every iteration." Resident items list in fixed prefix order , because the order <i>is</i> the cache. Each row carries a size and a tag: <code>cached</code> , <code>re-read</code> , <code>volatile</code> . <i>Compacted out</i> is the bridge to artifacts.
Long-term	Facts in one of four confidence states — verified · asserted · decaying · contradicted . Provenance beats recency : a passing verify outranks any later assertion. Editing a recalled fact does not overwrite it — the written fact stays revision 1, the correction becomes revision 2, and provenance still points at the run that wrote revision 1.
Wiki	Typed pages: <code>source</code> , <code>entity</code> , <code>concept</code> , <code>synthesis</code> , <code>decision</code> , <code>overview</code> . Ingest, not authoring is the entry point. A contradiction is flagged <i>at ingest</i> , not at query, as a card offering Adjudicate and Board card . <code>locus wiki lint</code> reports orphans, broken links, entities never given a page, and assertions with no source.
Artifacts	Plans, diffs, diagrams, images, recordings, walkthroughs — commentable, and a comment steers the agent that made it.
Mail	Threads between agents, the <code>mail wait</code> timeout, and handoff payloads.

ARTIFACTS — TWO REPRESENTATIONS, BECAUSE A HUMAN AND A MODEL WANT OPPOSITE THINGS

KIND	STORED	AGENT-FACING
<code>image</code>	WebP q80, longest edge capped at 2560	OCR text if the shot carries text ; otherwise downscaled to 1500px
<code>recording</code>	WebM as the browser produced it, capped by duration	extracted keyframes, never the clip
<code>diff</code> · <code>plan</code> · <code>walkthrough</code>	text in Postgres	the same text

An error dialog, a terminal capture, or a failing assertion is text wearing a screenshot's clothes. Only appearance — a layout, a rendering bug, a diagram — justifies sending pixels. Media has a retention policy (pruned with its run after 30 days unless linked to a PR or a task in Done); text artifacts are small enough to keep forever, and the trace depends on them surviving.

The kinds also split by **whether a human is meant to see them**. Review kinds (`plan` , `diff` , `diagram` , `image` , `recording` , `walkthrough`) reach the inbox when they need you. Reference kinds — `finding` and `payload` — **never** do; they are storage with a handle. Without that split the inbox fills with an agent's own scratch, and the one surface built to protect your attention becomes the one that spends it.

2.7 Dispatch — a durable queue, and the rate a human can absorb

A card being ready is not the same as a card being started. Dispatch is the queue between them, and it is *supervisor* state rather than UI state — caps, ordering, autorun, schedules, and the kill switch all survive a restart, because a queue that forgets is a queue that double-starts.

Caps come in pairs

A global cap on parallel agents and a per-project cap counted against the SAME pool, never in addition to it.

A run that would exceed either waits rather than starting thin.

Ordering is a named method

plan order · manual ·
unblocks-most · shortest-first
tie-break: longest-waiting

Preemption is off by default; when on, it pauses at the next ITERATION BOUNDARY.

Autorun is a per-project switch

on · off · suspended

There is no third setting and no per-task exception. A project whose verify pass rate falls under 60% SUSPENDS ITSELF and resumes on its own.

The review-slot model — the real constraint is not agents

A slot is one change you have not read yet. The median developer reviews four changes a week; eight concurrent agents produce thirty-one. Autorun drains at the rate you absorb, or it is just a faster way of generating a backlog.

Per project: review debt · pauses-at threshold · inbox budget per hour · change ceiling.

Stop all is reversible

Names its exact scope first, kills at the next boundary, optionally lets each agent spend 30s writing a handoff. Restorable for ten minutes.

Five classes never autorun, whatever the switch says

- 1 · anything touching migrations/** — append-only and irreversible in practice
 - 2 · any workflow containing a Gate node — the gate is the point; skipping it would be deleting it
 - 3 · anything over the change ceiling — past a reviewer's capacity
 - 4 · anything at project under the verify-pass floor 5
 - 5 · the first task of any plan
- Each is enforced at the point an autorun-originated run would enter the queue, and each is asserted independently.

Dispatch. Stopping never deletes branches, artifacts, or memory — it may request a handoff before stopping a run.

2.8 The inbox

With N agents running, your attention is the bottleneck. Everything that needs you arrives in **one** inbox — not scattered across panes, badges, and tiles you have to go looking at. The inbox is *you as a participant in the same mail system agents use*: locus ask from an agent, a Gate waiting on approval, a guardrail trip, a contradiction found at ingest, a workflow goal awaiting sign-off, a finished run needing review.

SILENCE IS THE DEFAULT

A session working normally produces nothing. The inbox tells you when something *needs* you, not when something *happened*. An item that documents no response is a notification, not inbox work — and notifications go to the activity feed behind the other pill. The inbox also keeps what it resolved, because a queue that empties leaves no record of how long a loop waited on you, which is the one number that says whether the human is the constraint.

PART III

Extending Locus

3.1 Extensions versus plugins — the distinction that governs everything else

Workshop has exactly two subgroups, and the split is not cosmetic. An **extension** is content Locus authors and materializes. A **plugin** is an executable integration Locus talks to. They have different contracts, different trust models, and different failure modes.

Extensions — content, authored in Locus

Nine editor targets, one three-pane editor

agents commands hooks
linters output-styles
rules skills base-context
Workflows

- stored in Postgres, versioned
- materialized fresh into every run
- Save is MANUAL — an extension edit is reviewed before it reaches a tree; a workflow edit autosaves

Plugins — executables, capability-negotiated

Three kinds, one common contract

cli_tool harness provider

- a trusted manifest + one executable
- JSON-RPC 2.0 over stdio, any language
- namespaced, negotiated capabilities
- bounded calls, DATA ONLY

A plugin declares and returns data; the first-party UI knows how to render it. No third-party UI code, ever. No plugin writes Locus persistence.

Core services — memory, mail, board, wiki, telemetry, tools — stay internal and are never plugins.

THE FIRST-PARTY ROSTER IS DELIBERATELY SMALL

PLUGIN KIND	FIRST-PARTY PLUGIN	CANONICAL ID
CLI Tool	GitHub CLI	gh
Harness	Pi	pi
Provider	OpenAI API / ChatGPT models	openai
Provider	Claude models	anthropic
Provider	OpenRouter	openrouter

That roster is an allowlist of *shipped* plugins, not a limit on the architecture. A user plugin with a valid trusted manifest is discoverable **without a Locus core source change**. A non-first-party plugin is either user-installed or absent — it is never silently treated as a built-in.

3.2 Writing an agent definition

An agent is a Markdown file with frontmatter. Not a graph, not a compile step. This is the single biggest simplification in the design: every harness already reads `name` + `description` frontmatter over prose, so materializing an agent is close to an identity transform plus a rename.

```
---
name: reviewer
description: Read-only critic; runs on task completion
harness: any           # or a specific one, when it matters
model_tier: high       # low | medium | high | xhigh – mapped in Settings → Harnesses
                        # a missing tier falls back UP, never down
task_class: research   # code | plan | research – sets retrieval depth; defaults to code
tools: [rg, gh, cargo] # allowlist, resolved against the marketplace index
skills: [audit-code]
rules: [no-secrets]
memory:
  scope: project       # agent | project – never cross-project
  write: propose       # none    cannot write memory at all
                        # propose writes land in the probation buffer for the keeper
                        # direct  writes straight to the store – the keeper only
---
```

You review code you did not write. Find the defect, not the style...

THE FIVE RULES THAT BIND AN AGENT DEFINITION

RULE	WHY, AND WHAT IT COSTS YOU
<code>tools</code> is enforced, not advisory	It is the container's install set <i>and</i> the allowlist. A tool absent from the list is absent from the image, so the agent cannot reach for it. Changing the list rebuilds <code>locus/agent-<hash></code> and invalidates the prefix cache; editing the prose below rebuilds nothing.
Immutable once referenced	An edit creates a new version; a run pins the version it used. Editing <code>builder</code> while <code>builder@4</code> is running does not change what that run is doing.
An unresolvable tool fails at save	Naming an entry absent from the marketplace index is rejected <i>at save</i> , with the name given — not at run time in a container.
Composition moves to workflows	An agent does not nest others structurally. It may call <code>locus agent invoke</code> at runtime if that is in its tool list — bounded in the core at depth 3 and fan-out 4 , which a workflow may lower and never raise. Depth 3 × fan-out 4 is at most 21 containers, which one machine survives; depth 4 is 85, which it does not.
Definitions live in Postgres	Import and export as <code>.md</code> , so a definition can be reviewed in a PR or copied between machines. Export must re-import to an identical definition.

PERMISSIONS ARE DECLARED IN TWO PLACES, AND THE SECOND MAY ONLY NARROW

SET ON	DECLARES	RULE
the agent definition	<code>tools</code> — the install set and the allowlist	the baseline; absent from the list means absent from the image
the workflow's Agent node	a <code>tools</code> subset, a <code>network</code> tier, and <code>write</code> scope for this role	subtracts only. A node can take the reviewer's write access away; it cannot grant a capability the definition did not have

Narrowing-only matters because the definition is what was reviewed. If a graph could widen, every workflow would be a place to re-grant privileges, and reading an agent would stop telling you what it can do. Enforcement is the container, not the harness.

3.3 The other seven extension types

One three-pane editor serves all of them — a type is a fixture switch on the component, not eight components. Frontmatter field kinds are *inferred from the field name* through one shared table: a name ending in `_tokens` renders `number`, `tools` and `roles` render `chips`, a name whose value set the registry can enumerate renders `select`, a boolean-named field renders `toggle`, everything else renders `text`.

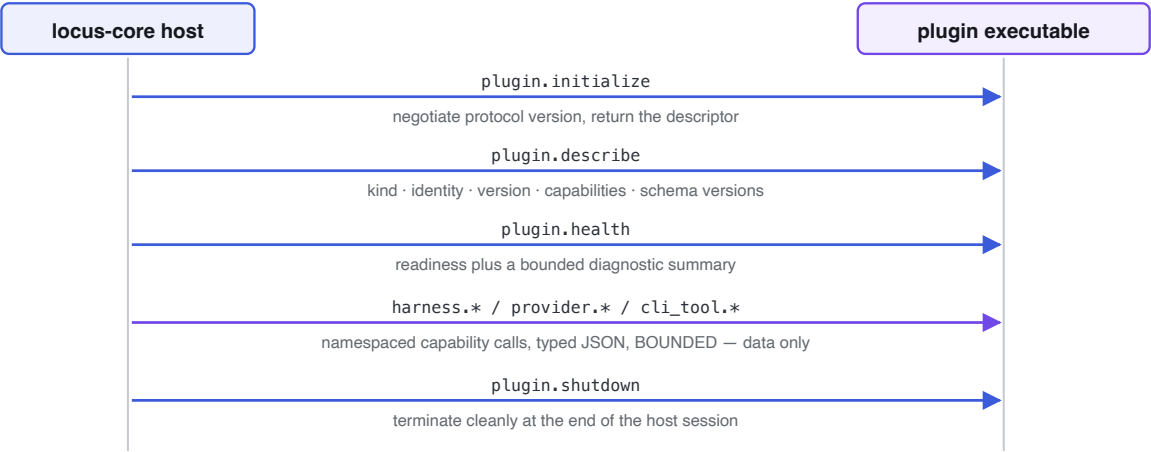
TYPE	THE INVARIANT A TEST HAS TO HOLD
<code>skills</code>	Lazy-loaded. A <code>budget_tokens</code> field refuses to materialize the skill above that cap. Downgrade inlines the description into base context and loses lazy loading.
<code>rules</code>	One glob per rule — a second match is a second rule, not a second glob. A <code>priority</code> field orders overlapping matches. The most downgraded type: concatenated into base context, firing on every file.
<code>base-context</code>	Exactly one per project. Native everywhere — it is the fallback every other downgrade lands in. Over budget means something upstream was downgraded and should be fixed <i>there</i> .
<code>commands</code>	Argument-taking prompt templates. A command with no args should be a skill instead. Downgrade materializes it as a skill, losing argument validation.
<code>hooks</code>	One event, a threshold, a timeout, and an on-error choice ending in exit 0 / log / continue. A hook that fails a run has turned an optimisation into an outage.
<code>output-styles</code>	Exactly one active per harness. A roles checklist decides which roles get which style. The largest single downgrade in the system when merged into base context.
<code>linters</code>	Native 0 / downgraded 0 <i>by design</i> — a human-and-CLI surface (<code>locus lint</code>), not materialized into any harness, so there is nothing to downgrade. A violation choice of <code>warn</code> or <code>fail</code> ; a rule nobody can fail is a preference.

HOOKS LOG AND INJECT. THEY NEVER THINK.
A hook cannot reuse the agent's LLM, and it fires on every tool call — so any model call inside one taxes the whole run. Two rules follow: the **injection path carries a 100ms timeout** and emits nothing on expiry, and the **logging path never touches the socket synchronously** — it appends to a local buffer and returns, with a background flush. Output must not begin with `{`, which some harnesses parse as JSON rather than as context.

3.4 The plugin contract

Every CLI-tool, harness, and model-provider plugin has a trusted manifest and one executable speaking JSON-RPC 2.0 over stdio, in any language. The manifest envelope is deliberately small.

```
# plugins/<id>/manifest.toml
protocol    = "locus.plugin.v1"
kind        = "harness"           # cli_tool | harness | provider
id          = "pi"
version     = "1.0.0"
executable  = "locus-plugin-pi"
capabilities = ["harness.session", "harness.materialize"]
permissions = ["model_catalog"]
```



The lifecycle runs before any kind-specific call. The executable is a separate process with no direct Postgres, Tauri, or filesystem authority outside the capability request it is handling.

WHAT THE HOST DOES, AND REFUSES TO DO

THE HOST...	CONSEQUENCE FOR YOU AS A PLUGIN AUTHOR
Rejects a protocol or version mismatch <i>before</i> dispatching any call	Get <code>protocol = "locus.plugin.v1"</code> right or nothing runs
Rejects malformed or required-unsupported capabilities	Declare what you actually implement; a harness must expose at least <code>harness.session</code>
Ignores unknown optional capabilities and preserves them for diagnostics	You can ship ahead of the host without breaking loading
Bounds every call and records structured diagnostics on timeout	A hung plugin degrades one call, not the run
Never infers behavior from a plugin <i>name</i>	Adding a capability must not require a core <code>match</code> on your id
Owns rendering, persistence, credentials, ACP normalization, and trust	Return data. Never attempt UI or a write path

3.5 Writing a harness plugin

A harness plugin is the largest of the three, because it has both halves of the contract: a **descriptor** that says where every extension goes, and **capabilities** that say how a session is launched and how its events arrive.

STEP 1 — THE DESCRIPTOR

The first-party Pi descriptor is the reference entry. Every field is mechanism; none of it is policy.

```
# harnesses/pi/pi.toml
name    = "pi"
binary  = "pi"
detect  = ["--version"]           # runs at IMAGE BUILD — its job is to fail the build
                                     # when the binary is missing, not to find one on your host
materializer = "pi/materialize"  # hooks and rules are TypeScript here, not files

[image]
base = "debian:bookworm-slim"

[launch]
argv = ["--print"]               # how to start ONE session in ONE run
tui  = false                     # REQUIRED false, enforced by the registry

[config]
home = "/locus/config"          # pi has no env override, so HOME is set

[telemetry]
source = "acp"                  # acp is the ONLY source

[models]
flag    = "--model"              # mechanism only — no model routing here
list_argv = []                  # how a model is passed to THIS harness
                                     # optional: how to ask it what it has

[layout]
agents    = { via = "dir", dir = "/locus/config/agents", format = "markdown+frontmatter" }
commands  = { via = "dir", dir = "/locus/config/prompts", format = "markdown+frontmatter" }
hooks     = { via = "plugin", emits = "/locus/config/extensions/locus-hooks.ts" }
linters   = { via = "dir", dir = "/locus/config/linters", format = "shell+markdown" }
output-styles = { via = "dir", dir = "/locus/config/output-styles", active = "brief-bright-gone" }
rules     = { via = "plugin", emits = "/locus/config/extensions/locus-rules.ts" }
skills    = { via = "dir", dir = "/locus/config/skills", format = "markdown+frontmatter" }
context   = { via = "file", file = "/locus/config/AGENTS.md" }
```

THE FAILURE THIS FILE EXISTS TO PREVENT IS SILENT

An omitted extension is **refused** at registration, with a message naming it. The worst failure mode in this whole class of tooling is a file that is "present, plausible, and loaded by nobody" — the tree is written, the run starts, and nothing loaded. Pi's own descriptor carries a comment recording exactly such a trap: *never write to a `commands` directory, because pi renames `<home>/commands` to `prompts` on startup, so writing the old name silently moves the tree.*

STEP 2 — PASS LOCUS HARNESS LINT

The lint refuses six things, and each corresponds to a way a capability quietly fails to arrive:

REFUSED	BECAUSE
an undeclared extension	an omission is indistinguishable from an unsupported capability
an unknown <code>via</code> strategy	there are exactly six, and core writes what they return

REFUSED	BECAUSE
<code>tui = true</code>	it breaks the one-session-per-run mapping everything depends on
a missing <code>[telemetry].source</code>	events would have nowhere to arrive from
a source other than <code>acp</code>	ACP is the only transport; the old four-source table is retired
a downgrade with no <code>weaker_than_native</code>	a capability you believe in but do not have is worse than one you know you lack

STEP 3 — MODEL ROUTING IS MECHANISM HERE, POLICY IN THE APP

An agent asks for `model_tier: high`. The descriptor supplies only how a model is passed and how to enumerate what exists. Which model `high` *means* lives in `core.settings`, keyed by harness and tier, and is edited in **Settings** → **Harnesses**. Three rules make it safe to leave half-filled:

- **A missing tier falls back UP, never down.** An agent asking for `xhigh` on a harness with no `xhigh` gets `high`. Falling *down* would quietly answer a hard question with a cheap model, and the result would look like a bad agent rather than a bad setting.
- **Unset means the harness's default.** With no mapping at all, no `flag` is passed — a harness that was never configured still runs. A new harness is usable the moment it is registered.
- **The resolved model id is recorded on the run**, not the tier — so spend and verify pass rate are attributable to what actually answered.

STEP 4 — DECLARE CAPABILITIES, NOT A NAME

The harness contract is capability-based rather than a fixed method list. The minimum for selection is `harness.session`; everything else is negotiable.

CAPABILITY	REQUIRED	COVERS
<code>harness.session</code>	YES	the minimum session capability needed for selection
<code>harness.launch</code>	OPTIONAL	launch mechanics beyond the descriptor's <code>[launch]</code>
<code>harness.events</code>	OPTIONAL	session events; all supported events map to the canonical ACP vocabulary
<code>harness.materialize</code>	OPTIONAL	the <code>plugin</code> strategy — return the files to write
model discovery, permissions, resume, checkpoints, usage	OPTIONAL	declared as needed; absence is recorded, never inferred

A harness may implement native ACP, an ACP adapter, or a materializer-backed configuration. Adding a capability must never require a new core match on the harness id — core validates the schema, routes the request, and preserves unknown optional data for diagnostics.

3.6 Writing a provider plugin

A provider is where a credential is *configured*, and it holds a **reference, not a key**. The record carries an identifier, an authentication method, an OS-keychain reference, an optional `base_url`, verification metadata, and a curated model catalog whose aliases are what every model selector displays.

```
#!/usr/bin/env python3 — a complete provider plugin, minus error handling
import json, sys

MODELS = [{ "id": "claude-sonnet-4", "alias": "Sonnet"},
           { "id": "claude-opus-4", "alias": "Opus"}]

def emit(req, result):
    print(json.dumps({"jsonrpc": "2.0", "id": req.get("id"), "result": result},
                    separators=(",", ":"), flush=True))

for raw in sys.stdin:
    req = json.loads(raw); m = req.get("method")
    if m == "plugin.initialize":
        emit(req, {"protocol": "locus.plugin.v1",
                  "capabilities": ["provider.models", "provider.verify", "provider.aliases"]})
    elif m == "plugin.describe":
        emit(req, {"protocol": "locus.plugin.v1", "kind": "provider", "id": "anthropic",
                  "version": "1.0.0", "schema_versions": {"plugin": "v1"},
                  "capabilities": ["provider.models", "provider.verify", "provider.aliases"],
                  "permissions": ["keychain_reference"]})
    elif m == "plugin.health": emit(req, {"ready": True})
    elif m == "provider.models": emit(req, {"models": MODELS})
    elif m == "provider.verify": emit(req, {"status": "unverified"})
    elif m == "plugin.shutdown": emit(req, {"stopped": True}); break
```

THE ONE THING A PROVIDER PLUGIN MUST NEVER DO

Return, log, cache, or accept a raw secret. Postgres stores the `keychain_reference` ; the key never enters a row, a repository, an artifact, a log line, or a container. The host broker resolves the reference *only* at the egress boundary. A provider plugin that handles a key has broken the single claim the whole credential model rests on.

3.7 Writing a CLI-tool plugin

Two artifacts, and they do different jobs. The **marketplace manifest** is the resolver's input — it validates an agent's `tools` list and supplies the one-line catalog entry. The **plugin executable** supplies install, verify, docs, and digest capabilities.

```
# index/gh.toml — the marketplace manifest
name      = "gh"
summary   = "GitHub's official command-line interface" # ~15 tokens; this is what
                                                    # reaches an agent's context

install  = { brew = "gh" }
verify   = "gh --version"
docs     = "docs/gh.md"
                                                    # fetched ONLY on demand, via
                                                    # locus tools docs gh

caps     = ["source-control", "github"]
```

Knowledge is JUST-IN-TIME

At session start the agent receives a CATALOG:

gh – GitHub's official CLI
rg – fast recursive search
cargo – the Rust build tool

Fifteen allowlisted tools ≈ 225 tokens, against
~3,000 for their full pages — and the difference is
recovered by any agent that actually reads one.

Installation is EAGER, deliberately

A tool absent from the allowlist is absent from
the image, because that is a PRIVILEGE BOUNDARY,
not a context decision.

Just-in-time applies to what an agent KNOWS,
never to what it CAN REACH.

A tool an agent cannot find is the second most
common reason a run stalls — check the project scope.

The same move the field made for MCP under three names — Tool Search Tool, Code Mode, the MCP-code-execution pattern — reached directly, as a catalog line and a `docs` verb.

BLURBS ARE A TUNING SURFACE, NOT BOILERPLATE

Anthropic reported a **40% cut in task completion time** from having an agent evaluate and rewrite tool descriptions. The index is git-backed, so a better description is a commit — and the event store already holds what it would be measured against: how often a tool was reached for, and whether those runs passed.

3.8 Writing a materializer

Only the `plugin` strategy needs an executable. It is called once per run with the extension set as JSON and **returns the files to write** — it never writes them itself.

```
→ materialize { harness, extension, root: "/locus/config",
                entries: [{ name, frontmatter, body }] }
← { files: [{ path, mode, content }] }
```

Three things fall out of returning data rather than writing it: a materializer is a **pure function** and therefore testable without a container, a buggy one **cannot escape the config tree** (core rejects any path that does not resolve under `root`, and writes nothing), and the same JSON is the fixture for an event-based "did this harness get its rules" test.

Pi's materializer is the reference because a generated TypeScript extension is the furthest a harness gets from copying a directory — it proves the contract at its hardest point:

```
def materialize(params):
    root = params["root"]
    extensions = params.get("extensions", {})
    files = []
    if "rules" in extensions:
        bodies = [e["body"] for e in extensions["rules"]] # order as given – sorted upstream
        files.append({
            "path": root + "/extensions/locus-rules.ts",
            "mode": 420,
            "content": TEMPLATE % json.dumps(bodies)})
    return {"files": files}
```

THE DETERMINISM CHECKLIST — EVERY ITEM IS TESTABLE, AND CI TESTS THEM

1. Materializing the same agent twice produces byte-identical trees; `diff -r` is empty. 2. A materializer that emits a timestamp *fails* a determinism test. 3. Generated file order and in-file list order are both sorted, asserted independently. 4. A returned path that escapes `root` is rejected and nothing is written. 5. The config tree is read-only for the run's lifetime — a mid-run write attempt fails. Nothing fails visibly when determinism breaks; the runs just get more expensive. That is why the check is in CI and cache rate is on the dashboard.

3.9 Trust and admission

PLUGIN KIND	ADMISSION REQUIREMENT
CLI tool	Minisign signature against a trusted public key held in Locus settings. A manifest and its binary must verify before entering the enabled catalog or an image. An unsigned or untrusted upload is <i>rejected</i> — read-only access is not a meaningful containment boundary for code executed during an image build.
Provider	Credentials remain OS-keychain references resolved by the host broker. Raw secrets never cross the plugin or persistence boundary.
Harness	A trusted descriptor loaded by the registry. The host owns run identity, container placement, ACP normalization, and policy; the plugin owns harness-specific mechanics only.

The admission rule is not open: untrusted executable plugins are refused. What remains undecided is the signature/trust-store *UX* for user plugins, which the marketplace installer owns.

PART IV

Workflows and governance

A workflow is a **loop toward a goal**, not a one-shot pipeline. It is also the team: its `Agent` nodes are the roster, each node carries a role, and the edges are the dependencies. There is no separate `Team` entity, and there is no `Job` — a workflow subsumes it, and running a single agent ad hoc needs no wrapper at all.

4.1 Six executable nodes

NODE	CONTRIBUTES
<code>Agent</code>	An agent definition, pinned by version , plus a <code>role</code> — the definition is reusable, the role is per-workflow. Carries this role's permission narrowing: a <code>tools</code> subset, a <code>network</code> tier, <code>write</code> scope.
<code>Task</code>	A unit of work, optionally sourced from the board.
<code>Loop</code>	The iteration construct: what repeats, and what resets between passes.
<code>Condition</code>	Deterministic routing. No model in the orchestration path — the graph decides, not an LLM.
<code>Gate</code>	A checkpoint: human, or another agent acting as reviewer.
<code>Verify</code>	The runnable success criterion. Required — <code>verify</code> is NOT NULL.

WHAT A `CONDITION` CAN SAY

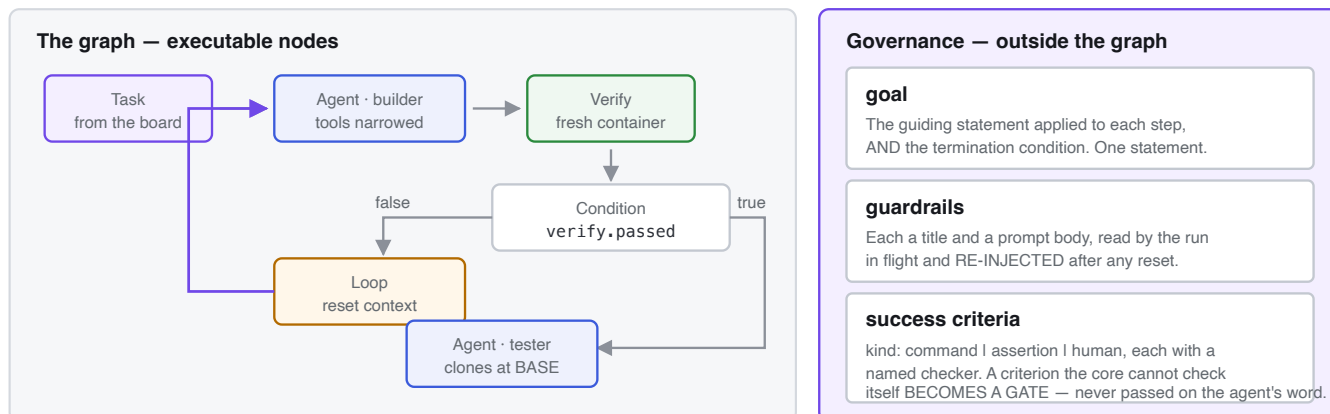
A small *total* expression language over facts the run already produced — no code, no model, no I/O, and no way to hang the orchestrator:

```
verify.passed · verify.exit_code · iteration · elapsed · tokens.used
events.count(tool_error) · events.last(kind) · artifact.exists(kind)
task.status · mail.pending

with == != < > <= >= · and or not · parentheses
```

Every operand is a column, so a `Condition` is a `WHERE` clause against the run — evaluable in microseconds and reproducible from stored events. Anything it cannot express is a `Gate`, which is to say a decision that deserved a person or an agent rather than an operator.

4.2 Governance — what sits outside the graph



Graph and governance are authored separately. Run state belongs to runs; the authoring surface holds no execution state at all.

WHERE VERIFY RUNS, AND WHY IT MATTERS

In a **fresh container from the agent's own image, on the run's branch** — never in the agent's container. Two reasons, and they are the same reason: the agent's container may already be gone, and its filesystem holds whatever the agent did outside git. *A check that passes only on the machine that made the change has verified nothing.* The command comes from the task or the node, the exit code is the result, and stdout plus stderr are captured as the evidence the board requires for Done.

4.3 Guardrails

Each is borrowed from a measured failure in an existing tool. Defaults apply to any agent run; a workflow may tighten or relax them, and they are what make leaving a loop unattended defensible.

GUARDRAIL	DEFAULT	WHY
<code>max_iterations</code>	8	Agents loop endlessly retrying the same broken approach without a hard stop
Forced reflection before retry	on	"What failed? What specific change would fix it? Am I repeating myself?" — reported to substantially cut stuck agents
Kill-and-reassign after 3 stuck iterations	on	A fresh context beats a confused one
Waiting ≠ idle	—	A run carries a <code>waiting</code> state with a reason — <code>ask</code> , <code>mail</code> , <code>debug-paused</code> , <code>gate</code> — and idle counts only time outside it. One mechanism, four callers. Without it every deliberate block reads as a stall
Idle detection	60s	No event on the run's stream for 60s while not blocked. The tile gets an idle icon and a toast fires <i>once per idle stretch</i> , never repeatedly
Wall-clock ceiling	none	Optional; a loop that cannot finish overnight should stop, not run to morning
Token budget	none — optional	When set, auto-pause at 85% and notify rather than draining silently

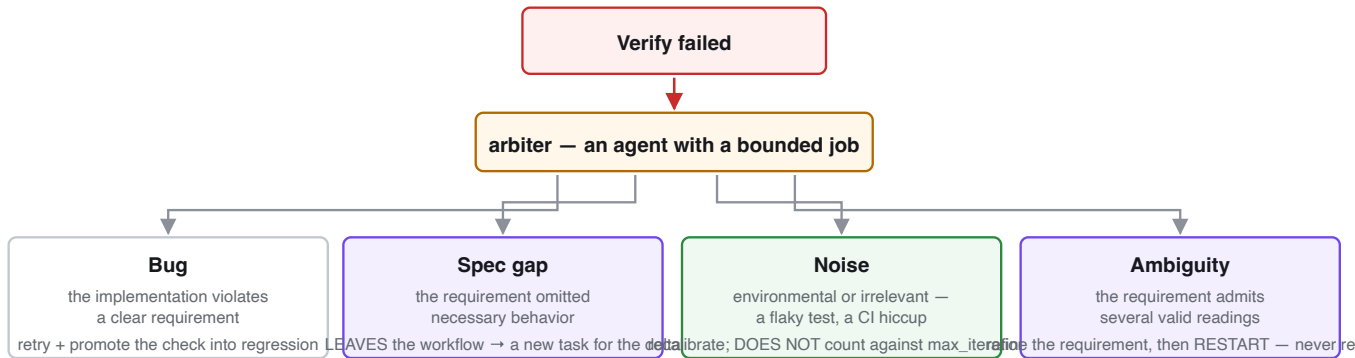
A BUDGET IS OPTIONAL; THE ACCOUNTING IS NOT

Token usage by itself reportedly explains 80% of the variance in agent task performance. If that holds even roughly, tokens are the strongest single predictor of whether a run went well — and **a run that passes verify on four times the tokens is a worse run wearing a green tick**. So every run carries usage whether or not a ceiling was set, agent trust is weighted by tokens per passing run, and the dashboard reports both.

Pause means the loop stops being fed, not that a process is frozen. The supervisor lets the current turn finish, holds before the next iteration, and notifies; the container stays up so its state is inspectable. `SIGSTOP` on a harness mid-request would leave sockets half-written and a model call in flight — a worse problem than the spend it saved.

4.4 When `Verify` fails, classify before retrying

Every guardrail above answers a failed iteration the same way: try again, with reflection, then give up. That is wrong for at least half of failures. Locus routes every failure through a four-way arbiter first, and each class has a different corrective action.



Two consequences worth having on purpose. Noise stops burning the iteration budget — three flaky failures otherwise kill a workflow at 8 iterations having attempted the work five times. And spec gaps and ambiguity *leave the workflow entirely*, which is the only path that reaches the thing actually broken.

The classification is a column on the iteration, so **spec-gap rate** and **ambiguity-detection rate** are queries — and a workflow that keeps producing spec gaps is visibly a planning problem rather than a builder problem.

4.5 Independence is structural, not requested

Role contamination is refused at compile time. One agent definition may not hold both the builder and the tester role in one workflow, and the reviewer may not be the implementer. This belongs in graph validation beside cycles and missing `verify` — not in a convention someone follows. A verifier that wrote the code inherits its assumptions, and a graph that quietly allows it produces reviews that agree with everything.

REGIME	MECHANISM	CATCHES	CANNOT CATCH
Independence-based	one agent implements, another writes tests — both from the requirement, neither seeing the other	contract violations, behavioral bugs	anything the requirement never said; shared model blind spots
Attention-based	one model directed into separate reviewer roles — security, architecture, product, QA	structural, product, and UX problems	it is not independent, so a shared misreading survives

Locus can enforce independence *structurally*, which is the part most setups only ask for politely: separate containers, separate sessions, no shared conversation, no shared short-term memory — all already true — plus the one the git model makes possible: **the tester clones the branch at base**, before the builder's commits. A verifier that cannot see the implementation cannot adopt its assumptions.

4.6 Worked example — builder, tester, reviewer

1. **Author the roster.** Drop three `Agent` nodes. Each pins a definition by version and carries a role. Narrow the tester's `tools` to read-plus-test; narrow the reviewer's `write` scope to none.
2. **Draw the dependencies.** An edge from builder to tester is the dependency. When the workflow creates board tasks, that edge *becomes* the tasks' `blocked_by` relationship — declared once, visually, rather than drawn on the canvas and re-entered on the board.
3. **Add `Verify`.** The runnable check. It is `NOT NULL`, so the graph will not compile without it.
4. **Write Governance.** The goal is the guiding statement *and* the termination condition. Add guardrail prompts and success criteria; anything the core cannot check becomes a gate that reaches your inbox with the evidence attached.
5. **Approve the goal.** That is the gate. Then it runs unattended: the tester starts the moment the builder's task completes, and the run stops on `max_iterations` when the goal is unreachable, with the live overlay showing which iteration tripped which guardrail.

THE RALPH LOOP IS A PRESET, NOT THE ONLY SHAPE

pick → *act* → *validate* → *commit* → *reset* is expressible in these nodes already: a `Loop` whose reset starts a fresh run in the same session, a `Verify` as the validate step, and Governance's goal as the termination condition. `locus ralph --goal ... --verify ...` runs one without opening the canvas, and dropping the preset onto the canvas **expands it into ordinary nodes** so it can be edited rather than configured. Two honest notes: it is *token-hungry by construction* — a fresh context every pass is the point — and it is *only as good as its verify*, because a loop iterating against a weak check converges confidently on the wrong thing.

The cost of deterministic routing, stated plainly: a workflow cannot re-plan itself mid-run. The lead is the graph you drew, not what a model decided this run, which is what makes a run reproducible and debuggable. If dynamic decomposition turns out to be needed, the fix is an agent that *authors a workflow* and submits it for goal approval — not a model in the execution path.

PART V

Operations

5.1 First-run setup

TASK	COMMAND
Install	<code>cargo fetch && pnpm -C apps/desktop install</code>
Build	<code>cargo build && pnpm -C apps/desktop build</code>
Test	<code>cargo test</code>
Test one module	<code>cargo test -p <crate> <module>::</code>
Lint	<code>cargo clippy --all-targets -- -D warnings</code>
Run locally	<code>pnpm -C apps/desktop tauri dev</code>
Validate every harness descriptor	<code>locus harness lint</code>

The `tauri` CLI is a devDependency of `apps/desktop` , not a global install — reach it through pnpm. Toolchain baseline: `docker 29` , `node v24` , `pnpm 10` , `cargo 1.97` .

BRINGING UP A PROJECT

- 1. **Create the project.** It owns the board, wiki, and memory across every repo it holds.
- 2. **Add repos** as *linked* (your checkout stays yours; Locus syncs with it) or *managed* (cloned from the forge, lives only inside Locus). A repo belongs to exactly one project.
- 3. **Configure harnesses** in Setup → Settings. The allow-list is ordered; exactly one row carries the agent default and it must itself be allow-listed. A harness with no working adapter is listed but not selectable.
- 4. **Set tier-to-model mappings** in Settings → Harnesses. Leaving them unset is safe: no `flag` is passed and the harness runs on its own default.
- 5. **Scope extensions and CLI tools.** Both are subtraction-only against Workshop's defaults. A tool change rebuilds the project image once.
- 6. **Write `base.md`** . Exactly one per project, with a token-budget meter.
- 7. **Seed the wiki.** `locus wiki ingest` reads the ADRs, specs, READMEs, and existing `AGENTS.md` files already in git, so the first keeper pass has a corpus rather than a blank store.

AN EMPTY MEMORY STORE DEGRADES CLEANLY

No catalog, no injection, no recall — agents run exactly as they would with no memory layer, which is the correct baseline. It need not stay empty, but nothing breaks while it is.

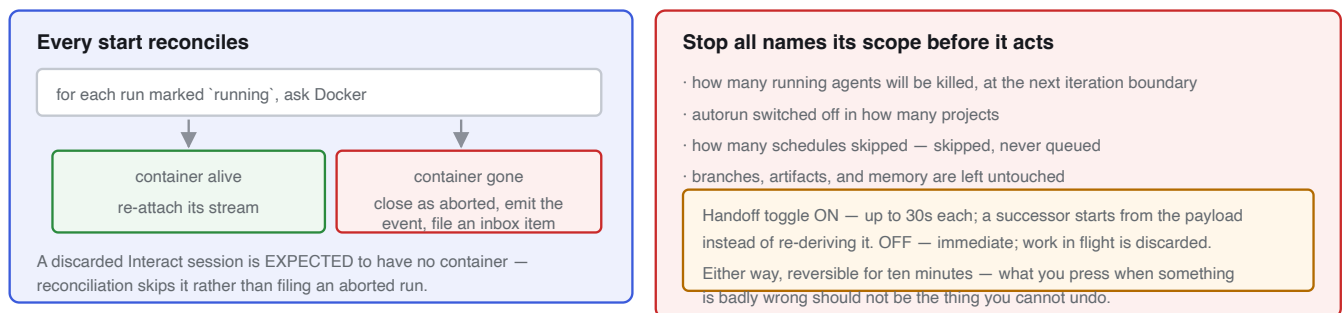
5.2 Backup and restore

With board, wiki, memory, and mail living *only* in Postgres, losing the volume loses work no reindex can rebuild. Event sourcing does not soften this — it sharpens it: the log itself lives only here, and the two carve-outs (embeddings, decay state) were never derived from the log, so a rebuild cannot restore them at any price.

```
locus backup           # dumps the nine schemas AND the artifact blob tree together
locus restore --drill  # restores into a SCRATCH database, asserts row counts against the source
locus rebuild         # regenerates every projection from log.entries – never touches the carve-outs
```

RULE	WHY IT IS A RULE AND NOT ADVICE
Nightly, and before every migration	Migrations are the one irreversible operation in the system
Seven dailies, four weeklies	Enough history to survive a problem noticed a month later
Backup covers both trees or neither	Text artifacts are rows; media is files under <code>/var/lib/locus/artifacts/</code> . Taking one and not the other restores a database full of paths to nothing
<code>--drill</code> is first-class, not documentation	A backup nobody has restored is a belief, not a backup

5.3 Reconciliation and Stop all



The two operations that decide whether the system is trustworthy across restarts and across panic.

5.4 The metrics that matter

Every one of these is already a column, so each is a query rather than new instrumentation. That is the point of normalizing events in the first place.

METRIC	READS	WHAT IT TELLS YOU
Cache rate	<code>usage.cache_read</code> ÷ <code>usage.input</code>	Below ~80% on a long session means something in front of the prompt is moving — and the run that did it is identifiable
Tool-payload offender ranking	<code>GROUP BY</code> over <code>tool_result</code> sizes	Which tool is putting the most bytes into history. Halving the biggest beats eliminating three small ones
Tokens per passing run	<code>usage</code> ÷ runs that passed verify	Separates a good run from an expensive one wearing a green tick
Verify pass rate	run rows	Under 60% <i>suspends a project's autorun automatically</i> and restores it when the number recovers
Spec-gap rate	arbiter classification per iteration	A workflow that keeps producing spec gaps is a <i>planning</i> problem, not a builder problem

METRIC	READS	WHAT IT TELLS YOU
Ambiguity-detection rate	arbiter classification per iteration	How often requirements admit two readings — the leading indicator for rework
Noise reclassified	arbiter classification per iteration	Iterations recovered that would otherwise have been burned by flaky checks
Agent trust	verify pass rate over the last 20 runs, discounted by guardrail trips, human-rejected artifacts, and tokens per passing run	Whether this agent has earned unattended work
Board dwell per column	<code>board.task_transitions</code>	Where the queue actually stalls. Usually not the agents

5.5 Troubleshooting

SYMPTOM	MOST LIKELY CAUSE	WHERE TO LOOK
A run starts but the agent behaves as if a skill or rule does not exist	Silent non-load — the tree was written but the harness reads a different path or filename suffix	<code>locus harness lint</code> , then the materialization smoke test: start a run with a canary skill and rule and assert the agent can see both
Runs got more expensive with no code change	Prefix instability — something mutable moved to the front of the prompt	Cache rate on the dashboard; the determinism check in CI; whatever injection point was added last
A run stalls partway	A tool the agent cannot find	Setup → CLI tools; the agent definition's <code>tools</code> list. A tool absent from the allowlist is absent from the image
<code>permission_request</code> events appearing	In bypass posture this is a misconfiguration <i>alarm</i> , not a prompt	The run's stored posture. In gated posture the same event is a legitimate blocked human action
Runs marked <code>running</code> that ended weeks ago	<code>locusd</code> crashed without reconciling	Restart — reconciliation closes them as <code>aborted</code> and files inbox items
A workflow dies at <code>max_iterations</code> having attempted the work five times	Flaky checks classified as <code>Bug</code> instead of <code>Noise</code>	The arbiter's classification column; recalibrate or quarantine the check
An agent idles at a breakpoint and trips the idle guardrail	The <code>waiting</code> state was not set	A paused debug session must suppress idle detection — one mechanism, four callers
A schedule fires but nothing runs	Overlap — the previous execution is still running	Overlap is skipped, never queued . A schedule skipping most firings raises a misconfiguration banner with <i>Widen the interval</i>
Autorun turned itself off	The project's rolling verify pass rate fell under 60%	Dispatch → Autorun. It resumes on its own; the UI distinguishes suspension from a manual off
A harness will not appear as selectable	No working adapter, or it is not in the project's allow-list	Workshop → Plugins → Harness, then Setup → Settings → Harnesses

PART VI

Current state

6.1 Milestones

MILESTONE	SCOPE	FEATURES	TASKS	STATE
M0	Specification and spikes	3	39	CLOSED
M0.5	Desktop UI on fixtures	12	268	CLOSED
M0.6	Desktop reconciliation	6	195	CLOSED
M0.7	Current mockup reconciliation	11	324	CLOSED
M1	Core runtime	14	270	CLOSED
M1.5	ACP agent panel, controls, plugin contract	5	64	ACTIVE
M2	Workspace — editor, LSP, search	3	57	QUEUED
M3	Coordination, memory, and mail	6	133	QUEUED
M3.5	Agent capabilities: debug and browser	3	56	QUEUED
M4	Workflow canvas	3	69	QUEUED
M5	Project management	5	127	QUEUED
M6	Automation and discoverability	3	46	QUEUED
M7	Forge providers	4	78	QUEUED
M8	Marketplace installer	1	11	QUEUED
Total		79	1,737	448 remaining

READ THE PHASE BOUNDARIES THIS WAY

M1 alone is a usable multi-agent runner without any of M2–M8. M4 is where it stops being a runner and becomes the product. Nine milestones at this scope is multi-year work, and the boundaries are drawn so that is survivable.

WHERE THE CURRENT WORK SITS

M1.5 is the focus: session controls and dispatch permissions are complete, the plugin contract is implemented and its first-party roster registered, and the remaining M1.5 items — the session research feed and the unified Agent Pane — are blocked on memory and planning-module landing first. Several later features have completed slices ahead of their milestone: the editor, mail, memory, repo manager, tool compaction, marketplace index, locus browse, LSP's CLI contract, and project search.

Three carry-forwards from M0 remain unexercised and are worth knowing about, because each affects trust in a feature that looks finished: real token usage has not been observed from a live run (if a harness reports nothing, spend must stay unknown, never zero); MergeView chunk revert, the three webview targets, and Cmd/IME behavior have not been exercised; and the Pi plugin has not been run end to end alongside a trusted user harness plugin.

6.2 Open decisions

These are unresolved *by design*, each owned by a spec. They are listed because an administrator reading a behavior that looks arbitrary deserves to know it has not been decided rather than being decided badly.

FEATURE	THE DECISION STILL OPEN
agent-definitions	Resolve <code>harness: any</code> at save time or at run start
workshop-plugins	The user-plugin signature and trust-store UX; first-party executable version pins. <i>The capability envelope itself is settled.</i>
board	The column-two label: Building or In Progress
guardrails	Whether the idle window scales by <code>task_class</code>
handoffs	Whether handoffs may cross projects
mail	Whether agents can mail the human directly, or must use <code>locus ask</code>
memory	The initial <code>importance</code> value before a memory is recalled
repo-manager	When a linked repo syncs: on demand, on a timer, or on git change
tool-compaction	One compaction threshold shared with <code>artifacts</code>
locus-browse	The recording duration cap, alongside the 30-day media retention policy
locus-debug	The supported language-adapter boundary; the tail is not covered
media-artifacts	The OCR-confidence threshold for falling back to the source image
marketplace-index	Curation versus selection — a vetted catalog with quality guarantees, or an open index where usage data ranks manifests. Locus already collects the usage data, which points at selection
workflow-engine	The arbiter's bounded model budget
planning-module	How to turn "I don't know what I want yet" into a goal
schedules	Timezone and DST behavior for cron expressions
dashboard-metrics	The cache-rate alert threshold, and what counts as a long session

6.3 Named risks

RISK	SHAPE, AND THE MITIGATION THAT EXISTS
A complete-looking requirement that is not	Every gate checks work against a requirement, so a requirement that omits a behavior produces a green run, a passing adversarial suite, a Done card with evidence, and a broken feature. <i>Nothing downstream can catch it — this is a limit of the shape, not a bug in it.</i> Mitigations: the reduction pass, specialization records, and making spec-gap rate a number somebody looks at.
Harness surfaces rot, in both halves	Structured events and config layouts are the harness's private business and change between releases. The failure is <i>silent</i> : the tree is written, the run starts, nothing loaded. Mitigation: a materialization smoke test per harness, on registration and in CI, with a canary skill and rule.
Prefix stability decays by accident	Nothing fails; the runs just get more expensive, and the cause is whatever injection point was added last. Mitigation: cache rate on the dashboard, determinism check in CI — a regression shows as a number rather than a slow drift nobody attributes.

RISK	SHAPE, AND THE MITIGATION THAT EXISTS
Postgres is irreplaceable	Board, wiki, memory, and mail live only there. Mitigation: backup and a restore <i>drill</i> from M1, not later.
Nested agents multiply containers	An agent built from three agents is four containers, each able to nest further. The cycle check, depth limit, and fan-out cap are not polish — without all three, one bad graph exhausts the machine.
DAP adapter coverage	The client is one implementation, but every language needs its own adapter baked into the image. Node, Python, and Rust are well served; the tail is not.
Webview inconsistency	Tauri ships no browser, so the UI runs on WKWebView, WebView2, or WebKitGTK. This is the genuine cost of Tauri over Electron. Fallback for WebKitGTK is a pinned newer version, not a framework change.
TUI-only harnesses are excluded	Deliberate, but real. A harness that insists on painting a full-screen interface cannot expose the one ACP conversation per run that Locus owns and renders.

APPENDIX A

The agent CLI

Inside its container an agent calls `locus ...`, which speaks to `/run/locus.sock`. This is the MCP replacement: a binary costs nothing until invoked; a server sits in context whether used or not. Every verb takes `--json`, compact and never pretty-printed, because the caller is usually a model.

MEMORY

<code>locus memory note add replace remove</code>	the bounded core tier; an over-cap write is an ERROR, never an eviction
<code>locus memory recall <query></code>	the store tier: structured + full-text + vector hybrid
<code>locus memory write --scope ... --why ...</code>	a fact, with provenance attached automatically
<code>locus memory forget <id></code>	traced back to its writer, then deleted

MAIL AND COORDINATION

<code>locus mail</code> <code>send list read reply drain</code>	agent ↔ agent, Rust-native
<code>locus mail wait</code>	blocks with a 15-minute default timeout, then returns empty. Sets <code>waiting</code> , so it never reads as idle
<code>locus ask <question></code>	escalate to the human; blocks, and appears in the inbox
<code>locus handoff <agent> --why ...</code>	transfer ownership: ends this session, opens the successor's on the same task and branch
<code>locus agent invoke <name>@<version></code>	a nested agent: own container, own clone. Depth 3, fan-out 4

BOARD AND WIKI

<code>locus task</code> <code>list show move assign comment</code>	the board, agent-driven. Done requires evidence
<code>locus wiki search read write history</code>	revisions attributed to the run
<code>locus wiki ingest <path url></code>	read a document into typed pages; flags contradictions at ingest
<code>locus wiki query <question></code>	synthesize an answer, filed back as a <code>synthesis</code> page so it is asked once
<code>locus wiki lint</code>	orphans, broken links, missing entities, unsourced assertions

REAL TOOLS, NOT A SHELL

<code>locus lsp</code> <code>def refs hover symbols rename diagnostics</code>	semantic navigation over <i>this run's clone</i> . An agent grepping for a symbol reads ten files to answer what one LSP call answers exactly — cheaper in tokens <i>and</i> correct on overloads, shadowing, and re-exports
<code>locus debug</code> <code>start break step stack vars eval</code>	DAP. The session lives in the core, keyed by run; the CLI holds nothing. <code>break --log</code> is a logpoint : it prints and keeps running
<code>locus browse</code> <code>open click fill press assert</code>	the project browser, one Playwright context per run. <code>assert</code> exits non-zero with structured JSON, so a <code>Verify</code> node can gate on it

<code>locus browse</code> <code>screenshot record console network</code>	screenshots and recordings land as artifacts automatically. <code>console</code> and <code>network</code> matter more than pixels — a failing UI usually explains itself in text
<code>locus svc up down <name></code>	start a project service container. Agents get no Docker socket
<code>locus tools list docs <name></code>	the allowlisted catalog, and a full page on demand
<code>locus lint [--changed --only NAME]</code>	this project's linters — the one extension no harness reads. Called by the agent before it commits, and by a workflow's <code>Verify</code> node; <i>never</i> from a hook, which would tax every tool call

ARTIFACTS AND RUN STATE

<code>locus artifact put <kind> <path></code>	publish a plan, diagram, or image for review
<code>locus artifact get <id> [--for-context]</code>	<code>--for-context</code> returns OCR text or a downscaled frame, never the raw bytes
<code>locus artifact comments</code>	feedback left on your artifacts, threaded — a comment steers the agent that made it
<code>locus run status artifacts</code>	this run's own state

TWO RULES THAT APPLY TO EVERY VERB ABOVE

Debugging advice is logpoints before breakpoints. `break` stops the world, and an agent that stops the world then has to remember to continue it — a stopped process nobody resumes looks exactly like a hung run. **Auto-waiting, not sleeps.** Playwright waits for actionability by default; an agent writing `sleep 2` is a flaky test being born.

APPENDIX B

Glossary

TERM	DEFINITION
ACP	Agent Client Protocol — JSON-RPC 2.0 over stdio, LSP-for-agents. The only agent-session transport in Locus.
Agent definition	Markdown plus frontmatter, versioned in Postgres. Immutable once a run references it.
Arbiter	The bounded agent that classifies a verify failure as Bug, Spec gap, Noise, or Ambiguity before anything retries.
Artifact	A structured deliverable — plan, diff, diagram, image, recording, walkthrough (review kinds) or finding, payload (reference kinds).
Autorun	A per-project tri-state switch — on, off, suspended — letting agents pick up their own work. Throttled by review slots, not by agent count.
Base context	The always-loaded instructions every session starts with. Exactly one per project; the fallback every downgrade lands in.
Canary token	A secret string in the materialized config that must never appear in output. If it does, the system prompt leaked.
Carve-out	A column that cannot be folded from the log because a model or a clock produced it: embeddings, and decay/confidence.
Dispatch	The durable queue between a card being ready and a run being started. Supervisor state, not UI state.
Downgrade	An extension materialized by a weaker mechanism than the harness natively offers. Must carry <code>weaker_than_native</code> naming the loss.
Extension	One of eight authored content types materialized per run into whatever a harness reads.
Gate	A checkpoint node — human, or an agent acting as reviewer. Also what an uncheckable success criterion becomes.
Handoff	A structured payload transferring ownership: goal, done, remaining, attempted, decisions, open. Not mail, not invoke.
Harness	An agent CLI, registered as a plugin. Pi is the only first-party one.
Interact session	A session with no board task: <code>open</code> , <code>promoted</code> , or <code>discarded</code> .
Keeper	The ordinary agent definition that consolidates memory on genuine project idle. The only thing with direct memory-write access.
Locator	<code>locus://<project>/...</code> — the one address space every navigation path resolves through.
Materializer	The code half of the harness contract. Six strategies; only <code>plugin</code> needs an executable.
Permission posture	<code>bypass</code> (default; a permission request is an alarm) or <code>gated</code> (a request blocks as a visible human action). Selected at dispatch, pinned on the run.
Plugin	An executable integration — <code>cli_tool</code> , harness, or provider — speaking JSON-RPC 2.0 over stdio. Data only, never UI.
Project	One or more repos, plus the board, wiki, and memory that span them.

TERM	DEFINITION
Provider	Where a credential is configured. Holds an OS-keychain <i>reference</i> , never a key.
Review slot	One change you have not read yet. The unit autorun is rate-limited in.
Run	One container lifetime = one ACP conversation. Bounded by the container exiting.
Session	A durable, named thread of work with one agent. Bounded by you closing it. The thing that survives a reset.
Specialization record	A domain's accumulated requirements (payments demands idempotency keys, ...), stored as a wiki <code>concept</code> page and injected into synthesis above a confidence threshold.
Stream position	<code>agents.events.stream_pos</code> — monotonic per project, assigned by the core rather than a Postgres sequence.
Verify	The runnable success criterion. <code>NOT NULL</code> on every workflow, and run in a fresh container from the agent's image on the run's branch.
Walkthrough	The artifact generated on session completion: a concise summary of what changed, screenshots and recordings inline. The actual answer to "six agents ran overnight, now what."

APPENDIX C

Where to read further

`PLAN.md` at the repo root is the architecture and the authority. `.specs/<feature>/` decomposes it into contracts and runnable tasks: a `spec.md` (purpose, the governing `PLAN.md` section, the contract, falsifiable acceptance criteria, dependencies, and what is genuinely open) and a `tasks.md` (numbered steps, each with a runnable `verify:`). A spec *cites* the section that governs it rather than restating it — one source for a decision, not two that drift.

IF YOU ARE WORKING ON...	READ
The plugin contract	<code>.specs/workshop-plugins/</code> , <code>.specs/harness-registry/</code>
Materialization and determinism	<code>.specs/materializers/</code> , <code>harnesses/pi/pi.toml</code> , <code>harnesses/pi/materialize</code>
Agent authoring	<code>.specs/agent-definitions/</code> , <code>.specs/workshop-revision/</code>
The runtime and isolation	<code>.specs/run-supervisor/</code> , <code>.specs/sandbox/</code> , <code>.specs/security/</code>
The store and event sourcing	<code>.specs/store/</code> , <code>.specs/event-store/</code> , <code>.specs/telemetry/</code>
Memory and knowledge	<code>.specs/memory/</code> , <code>.specs/wiki/</code> , <code>.specs/knowledge-revision/</code>
Screens and navigation	<code>.specs/design-revision/</code> , <code>docs/UI MOCKUP REVIEW.md</code> , <code>.specs/shell-revision/</code>
Project configuration	<code>.specs/setup-revision/</code>
Planning	<code>.specs/planning-module/</code> , <code>.specs/plan-revision/</code>
The board and sessions	<code>.specs/board/</code> , <code>.specs/manage-revision/</code> , <code>.specs/interact-sessions/</code>
Queueing, autorun, schedules	<code>.specs/dispatch-revision/</code> , <code>.specs/schedules/</code> , <code>.specs/guardrails/</code>
Workflows	<code>.specs/workflow-engine/</code> , <code>.specs/workflow-canvas/</code>
Agent capabilities	<code>.specs/lsp/</code> , <code>.specs/locus-debug/</code> , <code>.specs/locus-browse/</code> , <code>.specs/media-artifacts/</code>
Tools and the marketplace	<code>.specs/marketplace-index/</code> , <code>.specs/marketplace-installer/</code> , <code>.specs/tool-compaction/</code>
The evidence behind the three spikes	<code>spikes/01-sandboxed-harness/FINDINGS.md</code> , <code>spikes/02-editor-embed/FINDINGS.md</code> , <code>spikes/03-workflow-canvas/FINDINGS.md</code>
What is left to build	<code>TODO.md</code> — the unfinished-work index, with the dependency map

HOW LOCUS IS TESTED

Event-based. Every run normalizes into the same event vocabulary regardless of harness, capture path, or container — so a test is *"run this, assert these events appeared."* That works identically across the first-party Pi harness and any trusted user harness plugin, and needs no test-only instrumentation, because the substrate is the one telemetry already requires. Unit tests cover the pure parts; everything above them asserts on the event stream.

Compiled from `PLAN.md` and the 79 feature contracts under `.specs/` as of 25 August 2026, on branch `agent/todo-wave-m1-5-m3-5`. Where this guide and `PLAN.md` disagree, `PLAN.md` is the authority.